IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN XXXX-XXXX, Vol. 0, No. 0, pp. 1–31. DOI:XXXXXXXX

RIC: Randomize Invalid Coefficients to Mitigate Side-Channel Assisted Chosen-Ciphertext Attacks on ML-KEM

Junichi Sakamoto[1] and Kentaro Imafuku[1]

National Institute of Advanced Industrial Science and Technology,
Tokyo, Japan
sakamoto-junichi@aist.go.jp

Abstract. Module lattice-based key encapsulation mechanisms (ML-KEM) are susceptible to side-channel-assisted chosen-ciphertext attacks (SCA-CCAs) that exploit leakage from the re-encryption process during decapsulation. These attacks enable adversaries to recover secret keys with hundreds to thousands of oracle accesses, thereby posing a critical threat to the practical deployment of post-quantum cryptography. This paper presents RandInvalidCoeff, a novel and lightweight countermeasure that introduces randomness into the decryption function to mitigate SCA-CCAs. By randomizing invalid coefficients in the decrypted message polynomial, RandInvalidCoeff injects probabilistic errors into side-channel observations, significantly reducing the attacker's ability to perform reliable key recovery. This randomization slightly increases the decryption failure rate (DFR). Nevertheless, our analysis shows that strong resistance can be achieved with an acceptable DFR of approximately 2^{-80}. We provide information-theoretic and statistical analyses of the countermeasure to quantify the reduction in information leakage for plaintext-checking (PC) and decryption-failure (DF) oracle attacks. Furthermore, we performed PC- and DF-oracle attacks on an implementation with RandInvalidCoeff to validate the theoretical analysis, confirming that the proposed method achieves the expected countermeasure effect. The results demonstrate that the number of observations required for successful key recovery increases by a factor of more than eight, with only a few percent performance overhead compared to unprotected implementations.

Keywords: ML-KEM · Side-channel attack · Plaintext checking · Decryption Failure

1 Introduction

Classical public-key cryptosystems currently in widespread use, such as RSA and elliptic-curve cryptography, face the risk of being compromised by quantum computers in the near future. To address this issue, the standardization of postquantum cryptography (PQC) has become an urgent priority. In particular, the NIST-standardized module-lattice-based post-quantum key exchange (KEM) and signature schemes, module lattice-based key encapsulation mechanisms (ML-KEM) [oSN24b] and module lattice-based digital signature algorithm (ML-DSA) [oSN24a], were introduced in 2024. These cryptosystems, from the CRYSTALS family, were subjected to NIST's PQC competition as CLYSTALS-Kyber and CLYSTALS-Dilithium. In addition to these NIST-standardized lattice-based schemes, a hash-based signature scheme (SPHINCS+) and a code-based KEM scheme (HQC) have also been adopted. Among these, lattice-based schemes are considered the most promising owing to their balanced security and performance.

Licensed under Creative Commons License CC-BY 4.0.

Several national agencies have established guidelines for transitioning to PQC. For example, the U.S. Commercial National Security Algorithm Suite [(NS] recommends completing the PQC transition by 2033, at least for critical systems. Consequently, PQC is progressing rapidly from standardization to practical implementation. We have now reached a stage where the performance and security of practical implementation must be evaluated, rather than focusing solely on the theoretical aspects.

Side-channel attacks (SCAs) are a key consideration when discussing the practical security of cryptography. SCAs against PQC are a major topic, with numerous vulnerabilities reported, particularly in the FO transformation during KEM decapsulation. The FO transformation consists of two processes: decryption and re-encryption.

Decryption is the standard process for converting public-key encryption (PKE) ciphertext into plaintext using secret keys. In lattice-based cryptography, multiplication operations are performed between a secret key and a known ciphertext during decryption. Consequently, the secret key can be identified by directly observing traces of this operation using traditional SCA techniques [HHP+21, MBB+22]. In this study, we refer to SCA performed against decryption as *direct SCA*, meaning that the secret key can be derived directly from the observed traces.

Re-encryption verifies the input ciphertext by re-encrypting the plaintext obtained through decryption. Although re-encryption is essentially an encryption function that does not handle the secret key, it remains susceptible to SCAs because the decryption function in PQC-KEM is IND-CPA secure, making it inherently vulnerable to chosen-ciphertext attacks (CCAs). That is, the output of the decryption function must be treated as confidential information and not disclosed to an attacker. However, where side-channel observations are possible, the output of the decryption function can easily be leaked, making it vulnerable to CCAs. In other words, a side-channel attack on re-encryption can be interpreted as the construction of a decryption oracle through side-channel observations, which is subsequently used to perform chosen-ciphertext attacks against an IND-CPA-secure decryption function. We refer to this type of oracle-based attack as a side-channel-assisted CCA (SCA-CCA), or an indirect SCA, in this paper. Examples of SCA-CCA attacks against ML-KEM have been demonstrated in [TUX+23] and [HNPS24], where side-channel observations enable plaintext-checking oracle (PCO) or decryption-failure oracle (DFO) attacks. These attacks can recover the full secret key of unprotected implementations with only a few thousand oracle queries.

General countermeasures such as masking and hiding are effective against both direct and indirect SCAs for PQC. However, previous research indicated that high-order masking schemes are vulnerable to security flaws [WBD24, HNPS24, DNGW23]. Furthermore, even when no such flaws exist, sophisticated attack methods have been shown to break masking schemes using only a few hundred traces [HDH+25, ABB+24]. Although using higher-order masking can strengthen device security, it introduces significant performance overhead. Bos et al. [BGR+21] implemented a higher-order masking scheme in Kyber's decapsulation function and showed that the overheads of 1st-, 2nd-, and 3rd-order masking were approximately 2-, 19-, and 42-fold, respectively. These high overheads are impractical, necessitating the development of lightweight countermeasures.

The primary cause of this performance overhead is that the re-encryption function used for FO transformation employs numerous masking-unfriendly nonlinear operations. In contrast, the decryption function is nearly linear, allowing high-order masking to be implemented effectively. These observations raised the following research question: *Can the overall performance of decapsulation be significantly improved if the re-encryption function is efficiently protected using countermeasures other than masking?*

Intuitively, detecting specially crafted chosen ciphertexts used for SCA-CCA can enable efficient protection of the re-encryption function. Such an approach was proposed in [RCDB24] and [XPR+22] in the form of ciphertext sanity checks. Unfortunately, Ravi

et al. [RPJ+24] demonstrated that these checks can be easily bypassed by adding a multiple of the public key to the ciphertext, a technique known as a public-key mask (PKM). Ravi et al. [RCDB24] also proposed a different detection-based countermeasure, termed the message polynomial sanity check (MP-sanity check). Although the MP-sanity check can indeed detect chosen-ciphertext attacks, detection-based approaches still suffer from the unresolved issue of tamper reactions following detection. In particular, Ravi et al. [RCDB24] recommended updating or erasing the private key upon detecting a single attack attempt, which significantly weakens resistance to denial-of-service (DoS) attacks.

Our Contributions

- **RandInvalidCoeff**: We propose RandInvalidCoeff (RIC), a novel and lightweight countermeasure against SCA-CCA attacks. RIC introduces randomness into the decryption function such that any information obtained by an attacker through side-channel observations contains probabilistic errors. Since RIC is a form of hiding, it does not suffer from vulnerability to denial-of-service (DoS) attacks, unlike detection-based countermeasures. While RIC alone cannot completely prevent SCA-CCA attacks, combining it with other hiding techniques, such as shuffling, provides more effective and efficient protection against side-channel attacks than masking.
- **Theoretical analysis**: We present a theoretical analysis that evaluates the amount of information an attacker can obtain from side-channel observations and quantifies the effectiveness of RIC. Our analysis shows that, under reasonable parameter choices, the number of traces required for key recovery increases by a factor of eight or more. A drawback of the proposed method is an increased decryption failure rate (DFR); however, we show that sufficient resistance against SCA-CCA can be achieved with an acceptable DFR of approximately 2^{-80}.
- **Experimental validation**: We conduct experimental SCA-CCA attacks on RIC and demonstrate that its side-channel resistance is consistent with our theoretical analysis. Furthermore, we implement the proposed method on an x86 processor and evaluate its performance. Our results show that RIC can be integrated into higher-order masked implementations with only about a 1% performance overhead.

Software availability Omitted owing to anonymous submission.

Paper Organization The remainder of this paper is organized as follows. Section 2 reviews the preliminaries, including the basic structure of ML-KEM and existing SCA-CCAs, such as PCO and DFO attacks. Section 3 introduces the proposed countermeasure, RandInvalidCoeff, which randomizes invalid coefficients in the decryption process to randomize the occurrence of decryption failures (DFs) and mitigate SCA-CCAs. Section 4 presents a theoretical analysis of the proposed method, modeling the countermeasure, and quantifying its effect on information leakage. Section 5 provides experimental validation of the theoretical analysis, including both DFO- and PCO-based attack evaluations. Finally, Section 6 concludes the study.

2 Preliminaries

2.1 Notation

ML-KEM primarily uses three algebraic structures: the integer ring $\mathbb{Z}_q$, the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, and the polynomial vector R_q^k, where $n = 256$, $q = 3329$ is a prime number, and $k = \{2, 3, 4\}$ depends on the security parameter. In this paper, we

Algorithm 1 PKE.KeyGen (simplified)

Require: Randomness seed $d \in \{0,1\}^{256}$
Ensure: Encryption key ek_{pke}, decryption key dk_{pke}

1: $(\rho, \sigma) \leftarrow \mathrm{G}(d||k)$
2: $\mathbf{A} \in R_q^{k \times k} \leftarrow \mathrm{SampleUniform}(\rho)$
3: $\mathbf{e}, \mathbf{s} \in R_q^k \leftarrow \mathrm{SampleCBD}_{\eta_1}(\sigma)$
4: $\mathbf{t} \leftarrow \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$
5: **return** $\mathrm{ek}_{pke} = (\mathbf{t}, \rho), \mathrm{dk}_{pke} = \mathbf{s}$

Algorithm 2 PKE.Dec (simplified)

Require: Decryption key $\mathbf{s}$, ciphertext $c = (c_1, c_2)$
Ensure: Decrypted message m'

1: $\mathbf{u}' \leftarrow \mathrm{Decompress}_{d_u}(c_1)$
2: $v' \leftarrow \mathrm{Decompress}_{d_v}(c_2)$
3: $w \leftarrow v' - \mathbf{s}^\top \cdot \mathbf{u}'$
4: $m' \leftarrow \mathrm{Decode}(w)$
5: **return** m'

Algorithm 3 PKE.Enc (simplified)

Require: Encryption key $\mathrm{ek}_{pke} = (\mathbf{t}, \rho)$, message $m \in \{0,1\}^{256}$, randomness $r \in \{0,1\}^{256}$
Ensure: Ciphertext c

1: $\mathbf{A} \in R_q^{k \times k} \leftarrow \mathrm{SampleUniform}(\rho)$
2: $\mathbf{y} \in R_q^k \leftarrow \mathrm{SampleCBD}_{\eta_1}(r)$
3: $\mathbf{e}_1 \in R_q^k \leftarrow \mathrm{SampleCBD}_{\eta_2}(r)$
4: $e_2 \in R_q \leftarrow \mathrm{SampleCBD}_{\eta_2}(r)$
5: $\mathbf{u} \leftarrow \mathbf{A}^\top \cdot \mathbf{y} + \mathbf{e}_1$
6: $\mu \leftarrow \mathrm{Encode}(m)$
7: $\nu \leftarrow \mathbf{t}^\top \cdot \mathbf{y} + e_2$
8: $v \leftarrow \nu + \mu$
9: $c_1 \leftarrow \mathrm{Compress}_{d_u}(\mathbf{u})$
10: $c_2 \leftarrow \mathrm{Compress}_{d_v}(v)$
11: **return** $c = (c_1, c_2)$

use the following font conventions: lowercase (and, in some cases, uppercase) regular x for an element of $\mathbb{Z}_q$, lowercase Greek χ for an element of R_q, and lowercase bold $\mathbf{x}$ for an element of R_q^k. In the ML-KEM specification [oSN24b], some cases (e.g., e_2 and w) use Roman characters to denote polynomials; this study follows the same convention. We regard the three algebraic structures as an array of bits, an array of integers, and an array of polynomials, with indices i, j, k indicating the position of each element. For example, the notation $x[i]$ denotes the ith bit of the binary representation of an integer x, and $\mathbf{x}[i, j, k]$ denotes the kth bit of the jth coefficient of the ith polynomial in the polynomial vector $\mathbf{x}$.

2.2 ML-KEM

ML-KEM is a lattice-based post-quantum cryptosystem selected as the NIST's KEM standard. ML-KEM is an instance of an IND-CCA-secure KEM (Algorithm 1–3) obtained by applying a Fujisaki–Okamoto (FO)-like transformation to an IND-CPA-secure PKE (Algorithm 4–6). ML-KEM employs a variety of optimizations, such as the number theoretic transform (NTT) and ciphertext compression, which are not fully explained here. All polynomial multiplications are performed in the NTT domain. Further details are available in FIPS-203 [oSN24b].

2.2.1 ML-KEM PKE

Algorithm 1 describes ML-KEM's PKE key generation function, which takes a 256-bit random seed d as input and outputs an encryption key (public key) ek_{pke} and a decryption key (private key) dk_{pke}. The random seed d is first expanded to 512 bits using the hash function G. The functions SampleUniform and SampleCBD use the expanded randomness to generate random polynomials from a uniform distribution and a centred binomial distribution (CBD), respectively. By executing these samplings multiple times, we obtain a polynomial matrix $\mathbf{A} \in R_q^{k \times k}$ and polynomial vectors $\mathbf{e}, \mathbf{s} \in R_q^k$ with small coefficients $\in [-\eta_1, \eta_1]$. Finally, we compute $\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$, and set $\mathrm{ek}_{pke} = (\mathbf{t}, \rho), \mathrm{dk}_{pke} = \mathbf{s}$ as the public and secret keys, respectively. PKE.KeyGen is a deterministic function because both the uniform and CBD distributions are generated by SHA-3 family hash functions as pseudo-random number generators.

Algorithm 3 is the ML-KEM's PKE encryption function, which takes as input the encryption key ek_{pke}, a 256-bit message m, and randomness r, and outputs a ciphertext c. This function generates two LWE instances: $(\mathbf{u}, \mathbf{A}^\top)$ and $(\nu, \mathbf{t}^\top)$. Each i-th bit $m[i]$ of message m is encoded into each coefficient $\mu[i]$ of the polynomial μ as $\mu[i] = \lceil q/2 \rfloor \cdot m[i]$ and embedded into the LWE instance $(\nu, \mathbf{t}^\top)$. $\mathbf{u}$ and v form the ciphertext, but each is scaled by the compression function to reduce the ciphertext size. d_u and d_v represent parameters indicating the number of bits in the scaled coefficients.

Algorithm 2 shows the ML-KEM's PKE decryption function, PKE.Dec. PKE.Dec first decompresses the compressed ciphertext $c = (c_1, c_2)$ using a decompression function. The decompressed results $\mathbf{u}', v'$ no longer exactly correspond to the original $\mathbf{u}, v$ because some information is lost owing to the lossy compression function. Let $\Delta\mathbf{u}$ and Δv denote the compression errors such that $\Delta\mathbf{u} = \mathbf{u} - \mathbf{u}'$ and $\Delta v = v - v'$, respectively. Then, w in Line 3 can be expressed as follows:

$$\begin{aligned} w &= v' - \mathbf{s}^T \cdot \mathbf{u}' \\ &= (v + \Delta v) - \mathbf{s}^T \cdot (\mathbf{u} + \Delta\mathbf{u}) \\ &= \mu + (e_2 + \Delta v + \mathbf{e}^T \cdot \mathbf{y} - \mathbf{s}^T \cdot \mathbf{e}_1 - \mathbf{s}^T \cdot \Delta\mathbf{u}) \\ &= \mu + E \end{aligned} \tag{1}$$

The term $E \in R_q = e_2 + \Delta v + \mathbf{e}^T \cdot \mathbf{y} - \mathbf{s}^T \cdot \mathbf{e}_1 - \mathbf{s}^T \cdot \Delta\mathbf{u}$ is the noise component of w and the absolute value of each coefficient $|E[i]|$ is sufficiently small to be removed by the decoding function. The function $\mathrm{Decode} : R_q \to \{0,1\}^{256}$ decodes the input polynomial into a 256-bit integer by removing the noise components: it assigns zero to message bit $m[i]$ if the corresponding coefficient $w[i]$ falls within $[-\lceil q/4 \rfloor, \lceil q/4 \rfloor)$, and one otherwise. Decryption is *SUCCESS* ($m' = m$) if the norm $||E||_\infty < q/4$; otherwise, it is *FAILURE* ($m' \neq m$). The DFR is negligible—at least less than 2^{-128} for all parameter sets [oSN24b].

2.2.2 ML-KEM

The aforementioned ML-KEM PKE is secure only against chosen-plaintext attacks and remains vulnerable to CCAs. Similar to many other PQC KEM schemes, ML-KEM uses a variant of the FO transformation to convert the IND-CPA PKE into an IND-CCA KEM. Algorithms 4–6 show the key generation, encapsulation, and decapsulation algorithms for ML-KEM. The core of the FO transformation is the re-encryption process in the Decaps algorithm. Considering a chosen-ciphertext attacker, the input ciphertext c to the Decaps function can be maliciously crafted. To detect this, the FO transformation introduces a re-encryption step during the decapsulation procedure. First, the Decaps function decrypts the ciphertext c using PKE.Dec to obtain the decrypted message m'. Then, Decaps re-encrypts m' and compares the result c' with the original ciphertext c to detect invalid ciphertexts. If an invalid ciphertext is detected, a pseudo-random number is output to prevent the attacker from obtaining information regarding the secret key.

We clearly distinguish between *decryption failure (DF)* ($m' \neq m$) and *decapsulation failure* ($c' \neq c$). When an attacker modifies a ciphertext c and queries an invalid ciphertext c', a decapsulation failure always occurs; observing this failure does not provide any useful information. However, the occurrence of DF depends on how the ciphertext changes. If the attacker can observe DFs, this observation becomes advantageous. In a black-box model in which the attacker can observe only the function inputs and outputs, the FO-like transform completely hides the DFs behind decapsulation failures. However, in a more realistic gray-box model, side-channel observations may reveal DFs, which can be exploited in various attacks, as discussed in later sections.

Algorithm 4 ML-KEM.KeyGen

Require: Randomness seed d, z
Ensure: Encapsulation key ek, decapsulation key dk

1: $(\text{ek}_{pke}, \text{dk}_{pke}) \leftarrow \text{PKE.KeyGen}(d)$
2: $\text{dk} \leftarrow (\text{dk}_{pke}, \text{ek}_{pke}, \text{H}(\text{ek}_{pke}), z)$
3: **return** (ek, dk)

Algorithm 5 ML-KEM.Encaps

Require: Encapsulation key ek, randomness m
Ensure: Shared secret K, ciphertext c

1: $K, r \leftarrow \text{G}(m||\text{H}(\text{ek}))$
2: $c \leftarrow \text{PKE.Enc}(\text{ek}, m, r)$
3: **return** (K, c)

Algorithm 6 ML-KEM.Decaps

Require: Ciphertext c, decapsulation key $\text{dk} = (\text{dk}_{pke}, \text{ek}_{pke}, h = \text{H}(\text{ek}_{pke}), z)$
Ensure: Shared secret K

1: $m' \leftarrow \text{PKE.Dec}(\text{dk}_{pke}, c)$
2: $(K', r') \leftarrow \text{G}(m'||h)$
3: $\bar{K} \leftarrow \text{J}(z||c)$
4: $c' \leftarrow \text{PKE.Enc}(\text{ek}_{pke}, m', r')$
5: **if** $c \neq c'$ **then**
6: $\quad K' \leftarrow \bar{K}$
7: **end if**
8: **return** K'

2.3 Direct SCA on PKE.Dec

The PKE.Dec function first involves coefficient-wise multiplication of the secret key and ciphertext in the NTT domain. Because such multiplications heavily leak information regarding secret keys, they are often targeted by direct SCAs. Previous studies [KdG21, MBB+22] performed Correlation Power Analysis (CPA) on these coefficient-wise multiplications and successfully recovered the secret keys of the unprotected software implementations of Kyber-512 and Kyber-768 with only 80–200 traces.

NTT computation is another source of direct secret-key leakage. Xu et al. [XPR+22] proposed a simple SPA-like attack that exploits the leakage of the intermediate value after the (inverse) NTT computation, that is, $\text{INTT}(\langle \hat{\mathbf{s}}, \hat{\mathbf{u}} \rangle) = \langle \mathbf{s}, \mathbf{u} \rangle$. By choosing the polynomial vector $\mathbf{u}$ to have a single nonzero coefficient U, the attacker reduces the inner products $\langle \mathbf{s}, \mathbf{u} \rangle$ to a scalar multiplication $U \cdot \mathbf{s}$. Because the Hamming weight of each coefficient $(U \cdot \mathbf{s})[i]$ strongly depends on the secret key coefficient $\mathbf{s}[i]$, an attacker can determine $\mathbf{s}[i]$ through an SPA-like observation. Xu et al. [XPR+22] demonstrated that this technique recovers the full secret key of Kyber-512 with only a few traces in a non-profiling setting.

The CPA/SPA-like methods described above are non-profiling attacks. In a profiling setting where the attacker is permitted to collect reference traces and build templates in advance, the attacker can exploit these models to achieve significantly higher attack efficiency with fewer traces. Hamburg et al. [HHP+21] exploited a soft analytical side-channel analysis (SASCA) technique to recover the secret key efficiently via INTT-related leakage. They demonstrated that their method could recover Kyber's secret key with only a few traces, even under high-noise conditions and on masked implementations.

Huang et al. [HDH+25] proposed a sophisticated profiling attack for polynomial multiplication. They first reverse-engineered the multiplication operation on an ARM Cortex-M4 microcontroller and constructed a precise leakage model using a cycle-level linear regression technique. Using this model, they successfully attacked targets with weak leakage (stack-optimized implementations) using only tens of traces and even recovered first-order masked targets with a few hundred traces. Bock et al. [ABB+24] adopted the strategy of constructing an extremely precise leakage model by profiling over 100 million traces and successfully attacked a masked implementation using a single trace.

2.4 Indirect SCA (SCA-CCA) on re-encryption

IND-CCA-secure PQC KEMs, such as ML-KEM, are constructed from an IND-CPA-secure PKE using the Fujisaki–Okamoto (FO) transformation. The IND-CPA part of this

construction is vulnerable to CCAs, which are common targets of SCAs. By exploiting side-channel observations, an adversary can effectively instantiate a decryption oracle and indirectly mount an SCA by accessing it. The following subsections introduce representative oracles used for such indirect SCAs, also referred to as SCA-CCA.

2.4.1 Plaintext Checking Oracle (PCO)

Binary PCO A PCO is an oracle that, given a ciphertext modified by the adversary, returns whether the decryption result coincides with the original plaintext before modification. Let $c = \mathrm{PKE.Enc}(m, \cdot)$ be the ciphertext corresponding to the plaintext m, and let c' be a modified c. The PCO for adversarial query c' is given as follows [TUX+23]:

$$O(c'; m) = \begin{cases} 1 & \text{if } \mathrm{PKE.Dec}(c') = m, \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Such a PCO can be realized through side-channel observations, and many schemes, including lattice-based ones, are vulnerable to side-channel-assisted PCO attacks [UXT+22].

In the following, we describe the (binary) PCO-based attack against Kyber presented in [UXT+22]. The same attack procedure is applicable to ML-KEM. The adversary queries a sparse ciphertext $c' = (\tilde{\mathbf{u}}, \tilde{v})$, where $\tilde{\mathbf{u}} = \mathrm{Compress}(U \cdot \mathbf{I}_i)$ and $\tilde{v} = \mathrm{Compress}(V \cdot x^j)$. Here, $0 \le i < k$ and $0 \le j < 256$ denote the polynomial index and the coefficient index of the secret key under attack, respectively, $\mathbf{I}_i$ denotes the unit vector whose i-th component is 1 and all others are 0, and $U, V \in \mathbb{Z}_q$ are scaling and offset factors.

With such a ciphertext, Kyber's decryption function calculates $m' = \mathrm{Decode}(V - U \cdot \mathbf{s})$, where, due to the small norm of the secret key $\mathbf{s}$, an attacker can choose U such that all coefficients of $U \cdot \mathbf{s}$ are decoded to zero. Then, by adding an offset V to the j-th coefficient, only that coefficient is decoded as either 0 or 1, depending on the value of the corresponding secret key coefficient $\mathbf{s}[i, j]$.

As a result, the decoded message m' takes the value zero or non-zero. Consequently, the PCO in Equation (2) can be simplified to an oracle that merely determines whether $m' = 0$. From the side-channel perspective, a particularly powerful property is that an attacker can generate a sparse ciphertext that decrypts to $m' = 0$ without any knowledge of the secret key, by setting $U = 0$ and $V = 0$. The attacker can profile the computation using a zero ciphertext on the target device and construct a accurate template with $m' = 0$. Using this template, the attacker can classify the observed traces with high accuracy to determine whether they correspond to $m' = 0$. Ueno et al. [UXT+22] demonstrated that the secret key of unprotected Kyber-512 can be recovered with 3,072 side-channel traces using a deep learning-based template.

Parallel PCO Parallel PCO-based SCAs [TUX+23, RRD+23] have been proposed as extensions of the binary PCO-based SCA. Because the binary PCO reveals at most one bit of information regarding the secret key from a single observed trace, the attack effectiveness is limited. However, the parallel PCO enables the attacker obtain up to N bits of information from a single observation by querying a ciphertext for which N bits of m' are equal to $\mathrm{Decode}(V - U \cdot \mathbf{s}[i, j])$ whereas the remaining $256 - N$ bits are all zero. To realize such an attack, the observed traces must be classified not by simple binary classification but into 2^N classes. The t-value-based template [RRD+23] and machine learning-based classifier [TUX+23] have been shown to correctly classify traces into 2^N classes with nearly 100% accuracy for roughly $N < 9$. In principle, the number of oracle accesses required for a successful attack is reduced by a factor of N (i.e., it is expected to be $1/N$ times that of a binary PCO). The successful full-key recovery of Kyber-512 by using 190 [RRD+23] and 576 [RRD+23] traces have been reported.

2.4.2 Decryption Failure Oracle (DFO)

DFO-based attacks target the error term E in Equation (1) that arises during decryption. Because E contains information about the secret key $\mathbf{s}$, the attacker uses side-channel observations to establish inequalities in E (in practice, on its coefficients $E[i]$), thereby progressively narrowing the candidate space for $\mathbf{s}$. The attacker first adds a constant $\lceil q/4 \rfloor$ to the ith coefficient of the valid ciphertext c. Then, the occurrence of a DF depends on the sign of $E[i]$; consequently, the attacker can establish an inequality of the form $E[i] < 0$ or $E_i \geq 0$. The full secret key is recovered by collecting several thousand inequalities. Compared to PCO-based attacks, DFO-based attacks require a larger number of observations; nevertheless, they constitute an attack vector that should be considered.

Several techniques exist for key recovery from inequalities, and to the best of our knowledge, the state-of-the-art method is that of Hermelink et al. [HMS+23]. They reported full key recovery for ML-KEM-768 with approximately 7,000 inequalities. Moreover, they demonstrated that the attack remained effective even when the set of inequalities contained errors, albeit at the cost of collecting a larger number of inequalities. While PCO-based attacks rely on sparse ciphertexts that are relatively easy to detect, the ciphertexts used in DFO-based attacks are nearly indistinguishable from uniformly random ciphertexts.

2.5 Detection-based Countermeasures against SCA-CCA

In CCA settings, the attacker queries the decapsulation function with maliciously manipulated ciphertexts, which often results in ciphertexts or decrypted plaintexts lying far outside the normal distribution. Based on this observation, low-cost detection-based countermeasures against SCA-CCA have been proposed. However, these countermeasures are either circumvented in practice or trigger tampering reactions after an attack is detected. For example, discarding or updating the secret key upon a single detection would be the best choice from a security perspective; however, it would also render the system vulnerable to DoS attacks.

Decapsulation Failure (DcpsF) Check As mentioned in [RPJ+24], a DcpsF check is a straightforward approach for preventing SCA-CCA by treating each DcpsF as an attack and counting its occurrences. Because the probability that a valid ciphertext will lead to DcpsF is negligible, the occurrence of DcpsF can be regarded as evidence that the ciphertext has been tampered with by an adversary. Such failures can be naturally detected in line 5 of Algorithm 6, which is a strength of this countermeasure because it can be integrated with minimal modifications to the algorithm. Nevertheless, the design of tampering reactions remains a challenge, as mentioned earlier, and [RPJ+24] demonstrated that an adversary can circumvent this countermeasure and still perform SCA by leveraging valid ciphertexts.

Ciphertext Sanity Check Ravi et al. proposed a ciphertext sanity check (CT sanity check) [RCDB24] as a lightweight countermeasure for PCO-based SCAs. An adversary performing a PCO-based SCA uses sparse ciphertexts whose components are mostly zero, whereas normal LWE ciphertexts are uniformly distributed; rejecting such sparse ciphertexts can prevent PCO-based SCAs. This countermeasure is lightweight and implementation-friendly because it can be realized outside `ML-KEM.Decaps`, and the input ciphertexts remain in an unmasked form, even if `ML-KEM.Decaps` is masked. However, this countermeasure still suffers from the problem that tamper reactions are vulnerable to DoS attacks; [RPJ+24] reported that it can be bypassed by adding a public key to the ciphertext to break its sparseness and make it indistinguishable from uniform random.

Message polynomial sanity check The ciphertexts used in DFO-based attacks are obtained by modifying only a single coefficient of a valid ciphertext. Therefore, they

typically evade detection through CT sanity checks. A message-polynomial (MP) sanity check was proposed as a lightweight countermeasure against DFO-based attacks [RCDB24]. DFO attacks attempt to position the coefficient of the error term $E[i]$ in Equation 1 close to the decryption threshold (approximately $q/4$ or $3q/4$). Because it is negligible for a valid ciphertext to lie in these regions, a $w[i]$ value that falls near these thresholds can be treated as evidence of an attack. However, for masked implementations, an MP sanity check must be performed in the masked domain; to our knowledge, no practical implementations of a masked MP sanity check have been reported. Moreover, as explained earlier, the problem of tampering reactions remains, which renders the system highly vulnerable to DoS attacks.

3 Proposed method

Section 2.5 presents existing low-cost SCA-CCA countermeasures based on detection and highlighted their limitations. In this Section, we propose a novel, low-cost SCA-CCA countermeasure called the **RandInvalidCoeff** (RIC) algorithm. Upon detecting an invalid decryption result, RIC randomizes the results as tamper responses. This introduces probabilistic errors into the attacker's observations, thereby increasing the number of traces required for the key recovery. We further evaluated the impact of these induced errors from an information-theoretic perspective and demonstrated that the proposed method achieved a significant countermeasure effect under practical decryption error rates.

3.1 Randomized ML-KEM decapsulation with RandInvalidCoeff

Algorithm 7 presents the `PKE.Dec` function, which employs the proposed `RandInvalidCoeff`. The algorithm randomizes the decryption result m' by adding a random value r with the maximum value M_r to the message polynomial w. An inherent disadvantage of this countermeasure is the increase in the DFR. However, the effect of adding the random value r is mitigated by the decoding function if M_r is small. Hence, the negative impact of this countermeasure is typically negligible. By contrast, this randomization significantly affects chosen-ciphertext adversaries. Because some coefficients of the message polynomial w derived from a chosen ciphertext tend to lie close to the decoding threshold, the decryption result m' becomes highly sensitive to r, making successful attacks considerably more difficult.

`RandInvalidCoeff` is a function parameterized by two values, M_r and N_d, which, together with τ, are constants determined at implementation time. These three parameters determine the distribution of the random value r, sampled from a discrete uniform distribution with step size τ:

$$r \xleftarrow{\$} \{-M_r, -M_r + \tau, -M_r + 2\tau, \ldots, -\tau, 0, \tau, \ldots, M_r - \tau, M_r\}. \tag{3}$$

Here, M_r denotes the maximum absolute value of r and $N_d + 1$ is the number of possible values of r. The parameters M_r and N_d are selected such that $N_d \mid 2 \cdot M_r$.

3.2 Decapsulation Failure Rate (DFR) Analysis

The countermeasure effectiveness of the proposed method increases as the value of M_r increases; however, this also raises the DFR in the absence of attacks. Fig. 1 shows the effect of M_r on DFR using the tool in [pc] [1]. Compared with the original ML-KEM DFR [oSN24b] values of $2^{-138.8}$, $2^{-164.8}$, and $2^{-174.8}$ for parameters 512, 768, and 1024, respectively, the

[1] We estimated the DFR using the Kyber.py script, by modifying tail_probability(F, ps.q/4) to tail_probability(F, ps.q/4 $-M_r$) in Kyber_failure.py.

Algorithm 7 Randomized PKE.Dec (simplified)

Require: Decryption key $\mathrm{dk}_{pke} = \mathbf{s}$, ciphertext $c = (c_1, c_2)$
Ensure: Decrypted message m'

1: $\mathbf{u}' \leftarrow \mathrm{Decompress}_{d_u}(c_1)$
2: $v' \leftarrow \mathrm{Decompress}_{d_v}(c_2)$
3: $w \leftarrow v' - \langle \mathbf{s}, \mathbf{u}' \rangle$
4: $w' \leftarrow \textsc{RandInvalidCoeff}_{M_r,N_d}(w)$ ▷ Randomize invalid coefficients
5: $m' \leftarrow \mathrm{Decode}(w')$
6: **return** m'

7: **function** $\textsc{RandInvalidCoeff}_{M_r,N_d}(w)$
8: $\quad \tau \leftarrow 2 \cdot M_r / N_d$
9: $\quad$ **for** $i = 0$ to 255 **do**
10: $\quad\quad \zeta \xleftarrow{\$} \mathbb{Z}_{N_d+1}$ ▷ ζ is uniformly random from $\{0, 1, ..., N_d\}$
11: $\quad\quad r \leftarrow \zeta \cdot \tau - M_r \bmod q$
12: $\quad\quad w'[i] \leftarrow w[i] + r \bmod q$
13: $\quad$ **end for**
14: $\quad$ **return** w'
15: **end function**

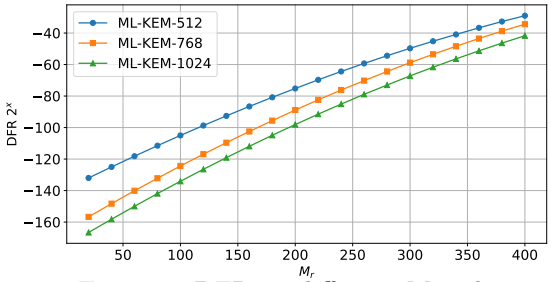


Figure 1: DFR on different M_r values

DFR worsens significantly as M_r increases. Nevertheless, even at $M_r = 200$, the DFRs for ML-KEM-768 and 1024 remain below 2^{-80}. We consider this DFR acceptable for many applications (discussed in Section 6.1) and show that M_r of approximately 200 exhibits a good countermeasure effect in the later section.

3.3 Performance overhead analysis

We implemented RIC in Algorithm 7 in a naive manner and measured its cycle count on an AMD EPYC 9R14 processor, obtaining approximately 3000 cycles. For comparison, the decapsulation function of the PQClean [PQC] speed implementation of ML-KEM-768 on the same platform requires approximately 133,000 cycles. Therefore, the overhead introduced by our proposed countermeasure in an unprotected implementation is about 2%. However, note that the naive RIC may produce negative intermediate values during the computation of r in line 11. Because negative integers are represented in two's complementary forms inside the processor, intermediate negative values can become a major source of side-channel leakage. Attacks exploiting this type of leakage, called determiner leakage, are well-known in the literature [DNGW23, SKL+20, HME24]. Therefore, RIC should be implemented in a side-channel-resistant manner using bit-slice or masking techniques to eliminate these leaks.

Next, we discuss higher-order masked implementations of RIC. Several studies have demonstrated that PKE.Dec can be compromised using only a few traces when an accurate

learning model is available [HDH+25, ABB+24], indicating that protecting PKE.Dec using the highest possible masking order to ensure security is essential. Accordingly, RIC should be implemented with a masking order consistent with that of PKE.Dec. Fortunately, the main operations in RIC (lines 11 and 12) involve only linear operations such as constant multiplications, subtractions, and additions, which can be computed independently for each share. Therefore, the computational cost of a higher-order masked RIC increases linearly with the masking order d.

The main challenge is splitting the random value ζ generated in line 10 into multiple shares. The first approach does not split ζ into shares but instead performs the RIC process only on the first share. Although this method cannot achieve d-probing security because the raw value of ζ appears as an intermediate value, it is still considered practical because recovering ζ from a single-side-channel observation is difficult. The second approach generates $d+1$ uniform random numbers $\zeta_i \xleftarrow{\$} \mathbb{Z}_{N_d+1}$ and treats $(\zeta_0, \ldots, \zeta_d)$ as Boolean shares of ζ, which are then converted through a Boolean-to-arithmetic conversion. This approach satisfies d-probing security; however, the range of ζ becomes $\mathbb{Z}_{2^{\lceil \log(N_d+1) \rceil}}$, and consequently, is no longer uniformly distributed. When $N_d + 1$ is a power of two, ζ becomes uniform. However, in this case, its distribution is biased toward positive values, requiring that M_r in line 11 be adjusted to $1.5 \cdot M_r$ to eliminate this bias. As this study did not evaluate such cases, the effectiveness of the countermeasures under these conditions requires further investigation.

We implemented RIC using the first approach and measured its cycle count on an AMD EPYC 9R14 processor over 100,000 runs. The measured average was approximately 3,000 cycles. Assuming that the additional cost introduced by RIC with the first approach is independent of the masking order d—as it adds randomness to only one share—we estimate the performance overhead by simply adding 3000 cycles to the cycle count of the masked Kyber (MK-KEM) implementation.

Table 1 lists the average cycle counts of the higher-order masked Kyber proposed by Coron et al. [CGMZ23], measured on an AMD EPYC 9R14 processor over 100,000 runs. For masking orders of one or higher, the performance overhead introduced by RIC remained below 1%, indicating that RIC enhances side-channel resistance while maintaining overall performance.

These results can also be interpreted differently. If RIC effectively reduces leakage from the ReEnc component, it may suffice to apply masking only to PKE.Dec while performing ReEnc with unmasked values without compromising side-channel security. Considering that PKE.Dec primarily consists of linear operations, whereas ReEnc involves mostly nonlinear computation; this strategy would result in significant performance improvements. The performance gains achieved by using this strategy are listed in the last row of Table 1. As the masking order increases, the degree of improvement also increases, with higher-order implementations expected to achieve a more than tenfold speedup. The level of security achieved under this strategy differs from that of a fully masked implementation. Nevertheless, because RIC can be combined with various lightweight SCA countermeasures, such as shuffling, jitter, or dummy operations, it would be possible to achieve a lightweight yet sufficiently secure implementation through such combinations.

4 Security Evaluation against SCA-CCAs

In this Section, we model DF-oracle-based and PC-oracle-based SCAs as a binary communication channel and hypothesis-testing problem, respectively, and evaluate the countermeasure effectiveness of RIC from an information-theoretic perspective.

Table 1: Average cycle counts for RIC using the first approach and the higher-order masked Kyber-768 proposed by Coron et al. [CGMZ23] measured on an AMD EPYC 9R14 processor over 100,000 runs (in thousands of cycles). The 0-th order implementation is the PQClean [PQC] clean ML-KEM-768 instance measured under the same conditions.

	Security order d					
	0	1	2	3	4	5
ML-KEM.Decaps	133	1655	3757	6809	10356	15253
PKE.Dec	20	64	93	132	168	370
RandInvalidCoeff	3	3	3	3	3	3
ReEnc	110	1590	3640	6691	10173	14814
RandInvalidCoeff overhead [%]	2.2	0.2	0.07	0.04	0.02	0.02
Perf. gain if ReEnc is unmasked	1	9.7	18.7	27.2	36.9	31.5

4.1 Security Evaluation against DFO-based SCA

4.1.1 Hermelink et al.'s BSC Model [HMS+23]

First, we recall the binary symmetric channel (BSC) modeling of the DFO-based SCA by Hermelink et al. [HMS+23] In their model, the attacker adds $V_e = \lceil q/4 \rfloor$ to the jth coefficient of the vth component of a valid ciphertext and queries the DF oracle, which returns whether a DF occurs for the ciphertext. Owing to the effect of V_e, the DF-oracle response depends only on the sign of the error term $E[j]$, allowing the establishment of either the inequality $E[j] \leq 0$ or $E[j] \geq 0$. Thus, we establish the following inequalities.

$$\begin{aligned}(e_2 + \Delta v + \mathbf{e}^\top \cdot \mathbf{y} - \mathbf{s}^\top \cdot \mathbf{e}_1 - \mathbf{s}^\top \cdot \Delta\mathbf{u})[j] &\lessgtr 0 \\ (\mathbf{e}^\top \cdot \mathbf{y} - \mathbf{s}^\top \cdot \mathbf{e}_1 - \mathbf{s}^\top \cdot \Delta\mathbf{u})[j] &\lessgtr (-e_2 - \Delta v)[j]. \end{aligned} \tag{4}$$

Letting $\mathbf{A} = (\mathbf{y}, -(\mathbf{e}_1 + \Delta\mathbf{u}))$, $\mathbf{x} = (\mathbf{s}, \mathbf{e})^\top$, and $b = -e_2 - \Delta v$, the above expression can be represented as a linear inequality in the unknown $\mathbf{x}$:

$$\mathbf{A}\mathbf{x}[j] \lessgtr b[j]. \tag{5}$$

These inequalities can be interpreted as indicating whether the current $\mathbf{A}\mathbf{x}[j]$ lies to the right or left of the point $b[j]$, when $\mathbf{A}\mathbf{x}[j]$ with a certain distribution is cut at $b[j]$. As $\mathbf{A}\mathbf{x}[j]$ can be approximated by a normal distribution $\mathcal{N}(0, 51^2)$, the average information obtained from the observation is given by $H_2(\Phi(b))$, where H_2 is the binary entropy function and Φ_0 is the cumulative distribution function of $\mathcal{N}(0, 51^2)$.

Because the attacker can choose only ciphertexts with a specific b, it is most efficient to query the oracle using only ciphertexts with $b = 0$, in which case, a single query yields at most one bit. Hermelink et al. demonstrated that by filtering and collecting only strong inequalities, key recovery of ML-KEM-512 was possible with 5,500 oracle accesses.

The above DFO-based SCA can be viewed as transmitting one bit—the direction of the inequality—to the attacker via a side-channel observation channel. Considering this as a BSC, errors in the inequalities arise from side-channel observations. Let the error rate of these observations be p. The information obtained by the attacker is at most $1 - H_2(p)$ bits, implying that the number of inequalities required for a successful attack increases by a factor of $\frac{1}{1-H_2(p)}$ compared to the error-free case. This relation can be regarded as close to a lower bound on the number of inequalities required for a successful attack; thus, the experimental results do not exactly match but still provide a good estimate. For example, they succeeded experimentally with 5,500 and 24,000 inequalities for $p = 1$ and $p = 0.8$, respectively. However, the lower bound on the number of inequalities for $p = 0.8$ given by the BSC is approximately 20,000.

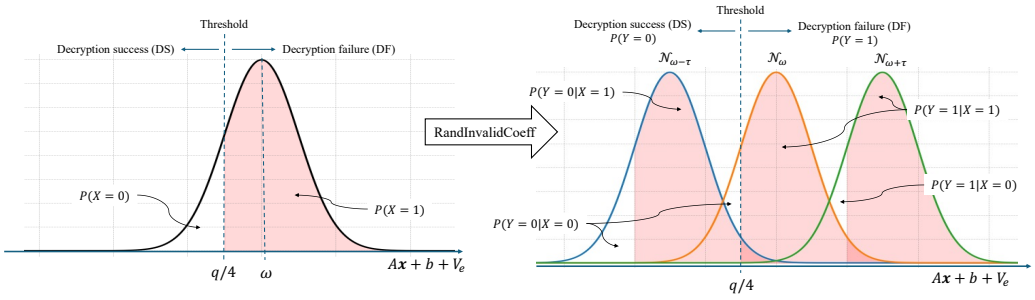


Figure 2: The distributions of X and Y for arbitrary b and V_e ($N_d = 2$).

4.1.2 Modeling RIC as a Binary Channel

Inspired by the BSC model of Hermelink et al. [HMS+23], we modeled RIC as a binary communication channel. Consider an attacker who sends the query $Q = \mathbf{A}\mathbf{x} + b + V_e$ to the DFO and obtains the binary response 0, representing decryption success (DS), or 1, representing decryption failure (DF). [2] We introduce two random variables X and Y. Let X denote the DF/DS distribution of the unprotected implementation and let Y denote the noisy version observed by the attacker, where the noise is caused by RIC. The distributions of X and Y are shown in Fig. 2 as an example, where $b + V_e = \omega$, $N_d = 2$. The left figure shows the distribution of X and the right figure shows the distribution of Y, randomized from X. Note that although the figure overlays three distributions, it is actually the sum of three distributions.

Using RIC, the distribution of Y is divided into $N_d + 1 = 3$ normal distributions, $\mathcal{N}_{q/4-\tau}$,$\mathcal{N}_{q/4}$, and $\mathcal{N}_{q/4+\tau}$, where $\mathcal{N}_\mu$ is the normal distribution with mean μ, $\mathcal{N}_\mu = \mathcal{N}(\mu, 51^2)$. The region to the right of the threshold is the DF-response region $P(Y = 1)$, and the shaded area within it corresponds to the correct response $P(Y = 1|X = 1)$. In contrast, the white area to the right of the threshold corresponds to the region that should originally return a DS response, $P(Y = 1|X = 0)$. This region introduces errors into the inequalities obtained by the attacker, rendering successful attacks more difficult.

When $\omega \neq \lceil q/4 \rfloor$, the prior probability X is not 1:1; therefore, the upper bound on the obtainable information, that is, $H_2(X)$, is less than one bit. In this case, since the error rate satisfies $P(Y = 1|X = 0) \neq P(Y = 0|X = 1)$, the channel becomes asymmetric. The channel capacity C is given as follows, depending on the prior probability X:

$$C = \max_{P(X)} I(X;Y). \tag{6}$$

The prior probability X also depends on ω and can be expressed as

$$P(X = 0) = \Phi_\omega(q/4), \qquad P(X = 1) = 1 - P(X = 0), \tag{7}$$

where Φ_μ denotes the cumulative distribution function $\mathcal{N}_\mu = \mathcal{N}(\mu, 51^2)$; The posterior probability Y depends on ω, τ, and N_d. Because it is simply the sum of the normal distributions above and below the decoding threshold,

$$P(Y = 0) = \frac{1}{N_d + 1} \sum_{\zeta=-N_d/2}^{N_d/2} \Phi_{\tau\cdot\zeta+\omega}(q/4), \qquad P(Y = 1) = 1 - P(Y = 0). \tag{8}$$

[2] Note that, in practice, the attacker sends the DFO a ciphertext $c + V_e$, not Q.

Next, we derived the joint probability $P(Y = 1, X = 1)$. Focusing on the normal distribution on the right-hand side of Fig. 2, its contribution to the joint probability is the region exceeding the decoding threshold, and its maximum is bounded by $P(X = 1)$. Therefore,

$$P(Y = 1, X = 1) = \frac{1}{N_d + 1} \sum_{\zeta=-N_d/2}^{N_d/2} \mathrm{clip}_{P(X=1)}(1 - \Phi_{\tau\cdot\zeta+\omega}(q/4)), \tag{9}$$

where

$$\mathrm{clip}_{th}(x) = \begin{cases} x & \text{if } x \leq th \\ th & \text{otherwise.} \end{cases} \tag{10}$$

The error rate can be derived as:

$$\begin{aligned} P(Y = 1|X = 0) &= \frac{P(Y = 1, X = 0)}{P(X = 0)} \\ &= \frac{P(Y = 1) - P(Y = 1, X = 1)}{P(X = 0)} \end{aligned} \tag{11}$$

and $P(Y = 0|X = 1)$ as well. Finally, the mutual information is calculated as:

$$\begin{aligned} I(X;Y) &= H(Y) - H(Y|X) \\ &= H_2(P(Y = 0)) - [P(X = 0) \cdot H(Y|X = 0) + P(X = 1) \cdot H(Y|X = 1)] \\ &= H_2(P(Y = 0)) - [P(X = 0) \cdot H_2(P(Y = 1|X = 0)) \\ &\qquad + P(X = 1) \cdot H_2(P(Y = 1|X = 1))]. \end{aligned} \tag{12}$$

We further define the ratio of the information obtainable before the countermeasure to the channel capacity as gain $G = 1/C$. Intuitively, the countermeasure requires the attacker to perform G times more oracle access to succeed.

4.1.3 Evaluation results

Fig. 3 presents the mutual information (Eq.(12)) for various countermeasure parameters M_r and N_d. As shown, the mutual information decreases monotonically as the attack parameter ω increases. When $\omega = 832 = \lceil q/4 \rfloor$, the mutual information is maximized for all countermeasure parameters M_r and N_d. Therefore, the channel capacity is $C = I(X;Y|\omega = 832)$ and it is optimal for the attacker to use ciphertext queries with $\omega = 832$.

Table 2 summarizes capacity C and its inverse G for various M_r and N_d. As shown in the table, increasing M_r and N_d significantly improves the effectiveness of the countermeasure. For some parameter sets, the attacker must perform over 50 times as many oracle accesses. Increasing M_r degrades the DFR, whereas increasing N_d has little effect on DFR. From the perspective of countermeasures against DFO-based attacks, using the largest possible value of N_d is recommended. However, in a later section, we show that a smaller N_d is more effective as a countermeasure against PCO-based attacks. Even when $N_d = 2$, setting M_r to approximately 200 requires more than 12 times as many oracle accesses for the attack, which we consider sufficiently effective as a countermeasure against DFO-based attacks.

4.2 Security Evaluation against PCO-based SCA

4.2.1 First Look at the Effect of the Countermeasure

First, we provide an intuitive explanation for the effectiveness of RIC against the PC-oracle-based SCA. Fig. 4 illustrates three coefficients of the message polynomial w under the

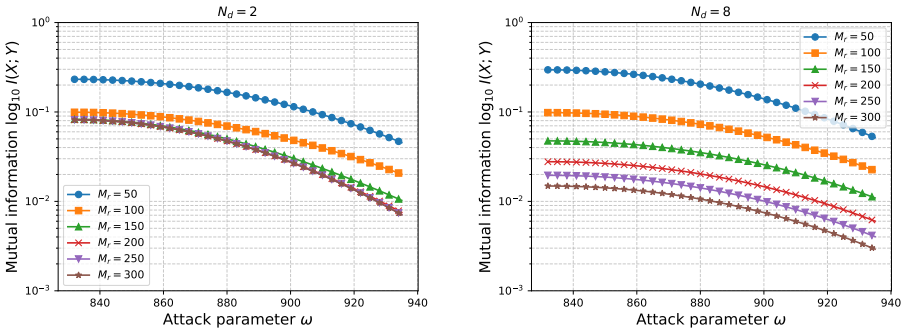


Figure 3: Estimated mutual information for various countermeasure parameters.

Table 2: Capacity C and its inverse (countermeasure gain) G for various M_r and N_d.

N_d	2					4				
M_r	50	100	150	200	250	50	100	150	200	250
C	0.232	0.099	0.083	0.082	0.082	0.259	0.091	0.049	0.035	0.031
G	4.31	10.07	12.07	12.23	12.23	3.86	11.03	20.61	28.41	32.53

N_d	8					16				
M_r	50	100	150	200	250	50	100	150	200	250
C	0.296	0.098	0.047	0.028	0.020	0.312	0.114	0.053	0.028	0.018
G	3.37	10.17	21.06	36.00	51.11	3.20	8.76	18.99	36.09	54.45

PCO-based SCA with $V = 416, U = 400$. The attacker now targets the 0th coefficient $w[0]$, which is shifted by $V = 416$; the other non-target coefficients remain unaffected by V. At this point, RIC introduces the following two effects as noise in the attacker's observations: The first effect is noise on the target coefficient $w[0]$. Let the secret key corresponding to the target coefficient $w[0]$ be denoted by $\mathfrak{s}$ (e.g., $\mathfrak{s} = \mathbf{s}[0, 0]$). If $\mathfrak{s} = -1$, then $w[0] = 816$, which is decoded as 0, would be observed as a DF without any countermeasure. However, the randomness introduced by RIC causes the value of $w[0]$ to fluctuate. Therefore, decryption succeeds with some probability, which injects errors into the attacker's observations. The second effect is noise on the non-target coefficients $w[i \neq 0]$. Although they all should be decoded to zero for the attack to succeed, the non-target coefficient $w[2]$ in the figure can be decoded to one because of the randomness introduced by RIC. Thus, when several nontarget coefficients are decoded to 1 (this occurs independently of the target coefficient), noise is introduced into the attacker's observations.

These two types of noises increase the number of oracle accesses required for a successful attack. Below, we model a PCO-based attack as a hypothesis-testing problem, and quantitatively evaluate the effects of RIC.

4.2.2 Attacker's model

We define a (binary) PCO-based attacker's ability using the following profiling model:

Profile phase First, the attacker creates a valid ciphertext c_0, which is the encrypted ciphertext of the message $m = 0$. The attacker then collects traces of the re-encryption process using c_0 and profiles the properties of the target (or reference) device to create a template. Although the attacker could generally profile the device using ciphertexts with $m \neq 0$, we focus on the specific case of $m = 0$ for simplicity without loss of generality. Invalid ciphertexts are not permitted for profiling. Because profiling typically requires significantly more traces than the actual attack phase, we assumed that profiling with invalid ciphertexts could be detected by the decapsulation failure counter.

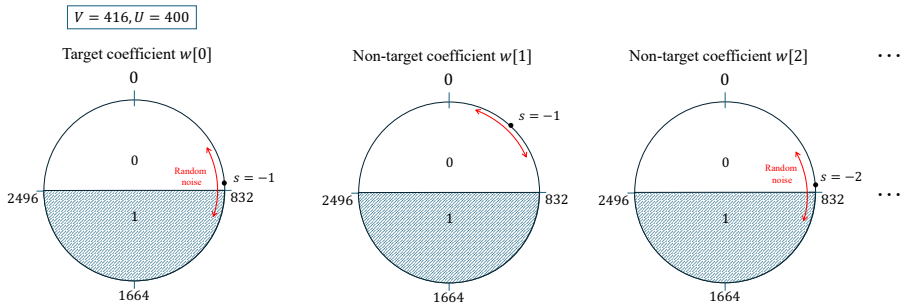


Figure 4: First three coefficients of w under the PCO-based SCA with $V = 416, U = 400$.

Attack phase The attacker sends an invalid ciphertext $\tilde{c_A}$ to the target device and determines whether the decrypted result $m' = 0$ by using the template. We define the attacker's chosen-ciphertext query $\tilde{c_A} = \mathrm{Compress}(c_A)$ as

$$\tilde{c_A} = \mathrm{Compress}(c_S + \mathbf{M}_{pk} + \epsilon) = (\tilde{\mathbf{u}_A}, \tilde{v_A}), \tag{13}$$

where $c_S = (\mathbf{u}_S, v_S)$ is a sparse ciphertext $(U \cdot \mathbf{I}_i, V)$ with $\mathrm{wt}(\mathbf{u}_S) = \mathrm{wt}(v_S) = 1$. This is known as a binary PCO attack. If countermeasures make a binary PCO attack difficult, parallel PCO attacks become even more difficult. $\mathbf{M}_{pk} = \alpha \cdot X^{\beta} \cdot (\mathbf{A}[\gamma], \mathbf{t}[\gamma]) = (\mathbf{u}_M, v_M)$ is a public key mask introduced in [RPJ+24], where $(\mathbf{A}[\gamma], \mathbf{t}[\gamma])$ is a part of the public key (the γ-th row), α and β are scaling and rotation factors. ϵ is a noise polynomial such that the sum of the absolute values of its coefficients, $\sum |\epsilon[i]|$, is sufficiently small to avoid affecting the decryption results. In the following, we assume $\epsilon = 0$ for simplicity.

4.2.3 Hypothesis test modeling

For clarity, we first model the case without a PKM, and then consider the effect of the PKM at the end of this section. Unless otherwise noted, the parameters of the ML-KEM-768 are used in the following explanation.

PCO-based attacks can be regarded as a hypothesis-testing problem between the null hypothesis H_0 and the alternative hypothesis H_1. Figure 5 shows how to distinguish whether the hypothesis $H_0 : \mathfrak{s} = -1$ or $H_1 : \mathfrak{s} = -2$ is true. In this figure, the attacker attempts to determine which hypothesis is true from the value of $w[0]$ using a chosen ciphertext $(U = 450, V = 0)$. For an unprotected implementation, $w[0]$ takes a value of 450 if $\mathfrak{s} = -1$ and 900 if $\mathfrak{s} = -2$; thus, one of the hypotheses can be completely rejected by observing a single PCO response. However, for the protected implementation with RIC, the attacker cannot conclusively reject either hypothesis from a single observation because the probability of DS is not zero under both hypotheses. If H_1 is true, the value $w[0]$ may also appear at positions located 150 units to the left and right of the center. Therefore, when $M_r = 150$ and $N_d = 2$, its position can be 750, 900, or 1050. Because the distribution is uniform, the probability of occurrence at each position is $1/3$. Consequently, $P(DS|H_1) = 1/3$. Thus, the attacker can compute the probability of DF under a given hypothesis, as shown in the figure. Let X be a random variable after decoding $w[0]$.

$$P(X = 1|H_i) = \frac{1}{N_d + 1} \sum_{\hat{\mathfrak{s}} \in H_i} \sum_{\zeta = -N_d/2}^{N_d/2} \mathrm{Decode}(V - U \cdot \hat{\mathfrak{s}} + \zeta \cdot \tau) \cdot P(\hat{\mathfrak{s}}|H_i), \tag{14}$$

where each hypothesis is treated as a set because it may include multiple key candidates $\hat{\mathfrak{s}}$.

Although $P(X|H_i)$ represents the distribution focusing only on the target coefficient, the observed result is affected by the noise introduced by the non-target coefficients. Let π

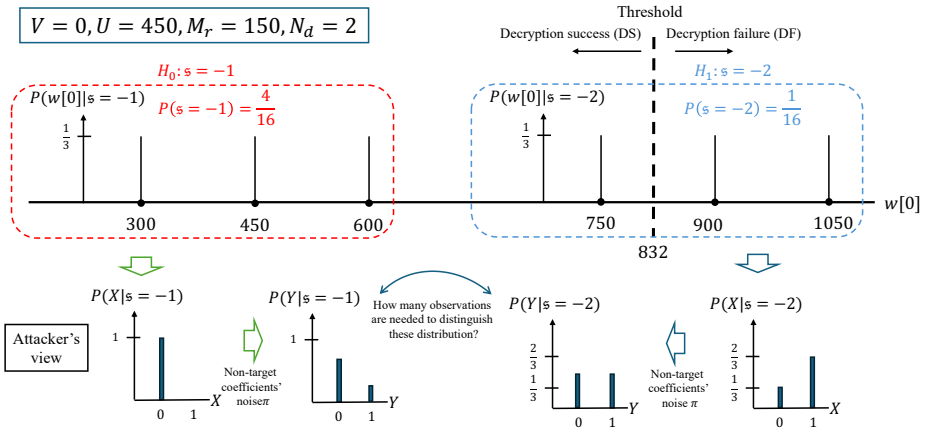


Figure 5: An illustration of how an attacker distinguishes whether $\mathfrak{s} = -1$ or $\mathfrak{s} = -2$ in the PCO-based attack.

denote the error rate caused by the non-target coefficients and let Y be a random variable representing the observation. Then, the probability of DF ($Y = 1$) under H_i is given by

$$P(Y = 1|H_i) = \pi + (1 - \pi) \cdot P(X = 1|H_i). \tag{15}$$

Using this probability together with the prior probabilities $P(H_i)$, we can compute the posterior probability of each hypothesis given N observations $y_{1:N}$ as follows:

$$P(H_i|y_{1:N}) = \frac{P(H_i) \cdot P(y_{1:N}|H_i)}{\sum_j P(H_j) \cdot P(y_{1:N}|H_j)} \tag{16}$$

For an attacker to succeed in a PCO-based attack, they must achieve a posterior probability close to one. Although increasing the number of observations N naturally increases the posterior probability, from a security evaluation perspective, we are interested in how large N must be for an attack to succeed. To evaluate this, we use the ratio of posterior probabilities, i.e., the posterior odds

$$\log \frac{P(H_0|Y)}{P(H_1|Y)} = \log \frac{P(H_0)}{P(H_1)} + \log \frac{P(y_{1:N}|H_0)}{P(y_{1:N}|H_1)}. \tag{17}$$

The second term on the right-hand side is the log-likelihood ratio (LLR) after N observations: Because the expected increment in the LLR per observation is given by the Kullback–Leibler divergence D_{KL}, the posterior odds after N_{H_0} observations (with H_0 being true) are

$$\log \frac{P(H_0)}{P(H_1)} + N_{H_0} \cdot D_{KL}(P(Y|H_0)||P(Y|H_1)). \tag{18}$$

Therefore, if we denote the target posterior odds by O_{goal}, the expected number of observations required to achieve O_{goal} under H_0 can be calculated as follows:

$$N_{H_0} = \frac{O_{goal} - \log \frac{P(H_0)}{P(H_1)}}{D_{KL}(P(Y|H_0)||P(Y|H_1))}. \tag{19}$$

Number of observations required to recover a secret-key coefficient To estimate the number of observations N_{obs} required for an attack using Eq. (19), the order in which to

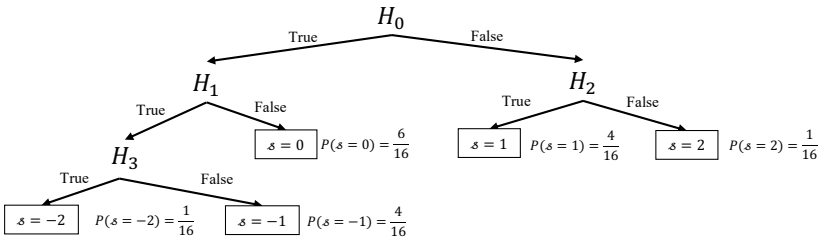


Figure 6: Hypothesis search path

reject the hypotheses selected from the given hypothesis set must be determined. Because a PCO-based attacker performs binary DF/DS classification to search for a key coefficient, it is reasonable to use hypotheses whose prior probabilities are close to 0.5. Specifically, when performing PCO-based attacks on ML-KEM-768, we employ the following eight hypotheses, where the alternative hypothesis to H_i is denoted as $H_{\bar{i}}$:

- H_0: target key $\mathfrak{s}$ is one of $\{-2, -1, 0\}$. $\longleftrightarrow$ $H_{\bar{0}}$: target key $\mathfrak{s}$ is one of $\{1, 2\}$.
- H_1: target key $\mathfrak{s}$ is one of $\{-2, -1\}$. $\longleftrightarrow$ $H_{\bar{1}}$: target key $\mathfrak{s}$ is 0.
- H_2: target key $\mathfrak{s}$ is 1. $\longleftrightarrow$ $H_{\bar{2}}$: target key $\mathfrak{s}$ is 2.
- H_3: target key $\mathfrak{s}$ is -2. $\longleftrightarrow$ $H_{\bar{3}}$: target key $\mathfrak{s}$ is -1.

The attacker continues to make observations until either hypothesis H_i or $H_{\bar{i}}$ reaches the target posterior probability O_{goal}. The expected number of observations required before rejecting the ith hypothesis is given by

$$N_i = \frac{P(H_i)}{P(H_i) + P(H_{\bar{i}})} \cdot N_{H_i} + \frac{P(H_{\bar{i}})}{P(H_i) + P(H_{\bar{i}})} \cdot N_{H_{\bar{i}}}. \tag{20}$$

By traversing the search path shown in Fig. 6, the expected number of observations required to recover a single secret coefficient at the target posterior odds O_{goal} is

$$N_{obs} = N_0 + P(H_0) \cdot N_1 + P(H_{\bar{0}}) \cdot N_2 + P(H_1) \cdot N_3. \tag{21}$$

Noise induced by non-target coefficients $w[i \neq 0]$ Owing to the randomness r introduced by the proposed method, the values of the nontarget coefficients $w[i \neq 0] = (U \cdot \mathbf{s}[j] + r)[i \neq 0]$ may exceed the decoding threshold and appear as DF. Let the probability of such a DF caused by non-target coefficients be π. Because π depends on the scaling factor U and one of the secret key polynomials $\mathbf{s}[j]$, we first make assumptions regarding the $\mathbf{s}[j]$ to evaluate π. Among the 254 coefficients $w[i \neq 0]$, those most likely to exceed the threshold are those with $\mathbf{s}[j][i \neq 0] = \eta_1$. Therefore, by ignoring the influence of coefficients in $\{-1, 0, 1\}$ (for ML-KEM-768), we calculate the probability θ that a single non-target coefficient with η_1 exceeds the decoding threshold as:

$$\theta = \frac{1}{N_d + 1} \cdot \sum_{\zeta = -N_d/2}^{N_d/2} \text{Decode}(|U| \cdot \eta_1 + \tau \cdot \zeta). \tag{22}$$

Considering that a secret key polynomial contains 32 coefficients with $|2|$ on average, we can approximate $\pi \approx 1 - (1 - \theta)^{32}$.

Infinity handling In Eq. (19), KL divergence is used to estimate the number of observations required for the attack. However, KL divergence diverges when zero-probability events

occur. For example, in Fig. 5, if $\pi = 0$, then $P(Y = 1|\mathfrak{s} = -1) = P(X = 1|\mathfrak{s} = -1) = 0$, causing the KL divergence to diverge, yielding $N_{\mathfrak{s}=-2} = 0$. This result is nonsensical because at least one observation is always required to recover the key. Intuitively, $P(X = 1|\mathfrak{s} = -1) = 0$ implies that observing $X = 1$ would immediately lead to the rejection of $\mathfrak{s} = -1$. However, because $P(X = 1|\mathfrak{s} = -2) = 2/3$, the probability of rejecting $\mathfrak{s} = -1$ using a single observation is low. Let p denote the probability of obtaining observation x that enables the rejection of one hypothesis. The probability that $X = x$ first occurs in the i-th trial follows a geometric distribution with an expected value of $1/p$. Thus, when KL divergence diverges, $N_{H_i} = 1/p$.

When divergence occurs for both hypotheses, we interpret this as a case with no observation errors owing to randomness and assign $N_{H_i} = N_{H_{\bar{i}}} = 1$.

Influence of PKM Because PCO-based attacks without the PKM technique can be easily detected by CT sanity check [RCDB24, XPR+22], we focused on evaluating the effectiveness of the proposed method when the attacker exploits the PKM. The plaintext polynomial w' calculated from a PKM ciphertext query $\tilde{c_A} = \mathrm{Compress}(c_S + \mathbf{M}_{pk}) = (\tilde{\mathbf{u}_A}, \tilde{v_A})$ is:

$$\begin{aligned} w' &= \mathrm{Decomp}(\tilde{v_A}) - \langle \mathrm{Decomp}(\tilde{\mathbf{u}_\mathbf{A}}), \mathbf{s}\rangle + r \\ &= v_S + \mathbf{e}[\gamma] + \Delta v_A - \langle \mathbf{u}_S + \Delta \mathbf{u}_A, \mathbf{s}\rangle + r \\ &= \underbrace{v_S - \langle \mathbf{u}_S, \mathbf{s}\rangle}_{\text{PC-oracle component}} + \underbrace{\alpha \cdot X^\beta \cdot \ \mathbf{e}[\gamma] + \Delta v_A - \langle \Delta \mathbf{u}_A, \mathbf{s}\rangle + r.}_{\text{Noise component}=\Delta w} \end{aligned} \tag{23}$$

As most elements of $\mathbf{c}_S$ are zero, the compression errors $\Delta v_A, \Delta \mathbf{u}_A$ originate from the PKM, hence, $(\Delta \mathbf{u}_A, \Delta v_A) \approx (\Delta \mathbf{u}_M, \Delta v_M)$, where $\Delta v_M = v_M - \mathrm{Decomp}(\tilde{v_M})$ and $\Delta \mathbf{u}_M = \mathbf{u}_M - \mathrm{Decomp}(\tilde{\mathbf{u}_M})$. Furthermore, because $||\mathbf{e}[\gamma]||_\infty \leq 2$ is small, the term $\alpha \cdot X^\beta \cdot \ \mathbf{e}[\gamma]$ is negligible[3] when $|\alpha|$ is small. Thus, we assume that the noise component can be approximated as

$$\Delta w \approx \underbrace{\Delta v_M - \langle \Delta \mathbf{u}_M, \mathbf{s}\rangle}_{r_{pkm}\text{: Noise component induced by } \mathbf{M}_{pk}} + r. \tag{24}$$

This indicates that, in addition to the noise r induced by RIC, another noise component r_{pkm} is induced by the PKM $\mathbf{M}_{pk}$, which remains unknown to the attacker. Note that r_{pkm} is not true randomness. Although r is truly random and changes with each oracle access, r_{pkm} remains constant as long as the attacker uses the same $\mathbf{M}_{pk}$; therefore, it acts as an unknown fixed offset rather than randomness.

Given a uniform random $\mathbf{M}_{pk}$, variable r_{pkm} follows the distribution $\mathcal{D}_{pkm}$:

$$\mathcal{D}_{\mathrm{pkm}} = \frac{2^{d_v}}{3328 + 2^{d_v}} \cdot \sum_{i=-3328/2^{d_v+1}}^{3328/2^{d_v+1}} \mathcal{N}(i, 236k). \tag{25}$$

Appendix A presents the derivation of $\mathcal{D}_{pkm}$.

Even when considering the effect of the PKM, the procedure for determining N_{obs} is almost identical to that without a PKM. However, to account for the additional noise r_{pkm}, it is necessary to replace $P(X = 1|H_i)$ in Eq. (14), and θ in Eq. (22) with Eq. (26) and Eq. (27), respectively:

$$\begin{aligned} P(X = 1|H_i) = \frac{1}{N_d + 1} \sum_{\hat{\mathfrak{s}} \in H_i} \sum_{\zeta=-N_d/2}^{N_d/2} [(1 - \Phi_{pkm}(\lceil \frac{q}{4} \rfloor - V + U \cdot \hat{\mathfrak{s}} - \zeta \cdot \tau)) \\ + \Phi_{pkm}(-\lceil \frac{q}{4} \rfloor - V + U \cdot \hat{\mathfrak{s}} - \zeta \cdot \tau)] \cdot P(\hat{\mathfrak{s}}|H_i), \end{aligned} \tag{26}$$

[3] Because smaller noise components are advantageous to the attacker, we consider that this assumption to yield a more conservative security evaluation.

$$\theta = \frac{1}{N_d+1} \cdot \sum_{\zeta=-N_d/2}^{N_d/2} [(1-\Phi_{pkm}(\lceil \frac{q}{4} \rfloor + |U| \cdot \eta_1 - \zeta \cdot \tau)) + \Phi_{pkm}(-\lceil \frac{q}{4} \rfloor + |U| \cdot \eta_1 - \zeta \cdot \tau)], \tag{27}$$

where Φ_{pkm} denotes the commutative distribution function of $\mathcal{D}_{pkm}$.

4.2.4 Evaluation results

We evaluated the concrete values of N_{obs}, which depend on several variables: O_{goal}, M_r, N_d, U and V. First, we set the attacker's target odds to $O_{goal} = \log\frac{0.99}{0.01}$ in the following evaluation: Next, M_r and N_d are the countermeasure-side parameters, whereas U and V are the attacker-side parameters. Assuming that the attacker can choose the optimal U and V for any fixed M_r and N_d, we are interested in the minimum N_{obs} under these conditions for the worst-case security evaluation. Therefore, the minimum number of observations required to recover a single secret key coefficient for a given pair of countermeasure parameters M_r and N_d is defined as follows:

$$N_{obs}^{min} = \min_{U,V} N_{obs}. \tag{28}$$

Evaluation results without PKM Fig. 7 shows the concrete values of N_{obs}^{min} for various countermeasure parameters M_r and N_d under the condition without a PKM. As M_r increases, all graphs exhibit a stair-like behavior, resulting from the discreteness of V. The larger N_{obs}^{min} value in the 512-parameter setting, compared with that in the 768/1024-parameter setting, results from the effect of η_1; the greater inherent entropy of the secret key coefficients in the 512-parameter setting requires more observations to determine the key. When M_r is small, N_{obs}^{min} is approximately 2.5, indicating that the countermeasure has little effect, since the entropy of the secret key coefficients is 2.03 bits for $\eta_1 = 2$ and 2.33 bits for $\eta_1 = 3$. As M_r increases, the countermeasures become more effective. For $M_r > 300$, the required number of observations increases by more than a factor of 10 in the 512 setting and by approximately a factor of seven in the 768/1024 setting. However, an M_r value of approximately 210 was considered reasonable for operating at a practical DFR of approximately 2^{-80}. In this case, the countermeasure provides at most a five-fold improvement, even under favorable estimation.

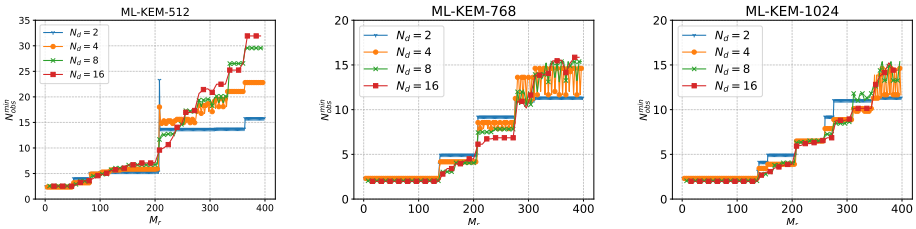


Figure 7: Evaluation results of N_{obs}^{min} (the number of observations required to recover a secret key coefficient) without the PKM technique.

Evaluation results with PKM We focus on the effectiveness of the countermeasure, particularly when the attacker uses the PKM technique, because attacks without PKMs are easily detected by CT sanity checks [RCDB24]. Fig. 8 shows the concrete values of N_{obs}^{min} for various countermeasure parameters M_r and N_d with a PKM. The noise introduced by the PKM makes the graph smoother and increases $N_{obs}^{\min}$ overall. For the

ML-KEM-768 setting, when $N_d = 2$ and $M_r = 210$, we obtain $N_{obs}^{\min} = 16.42$, which simultaneously achieves DFR $< 2^{-80}$ and an improvement of more than a factor of eight. It should be noted that this evaluation assumes the attacker randomly selects PKMs for each query.

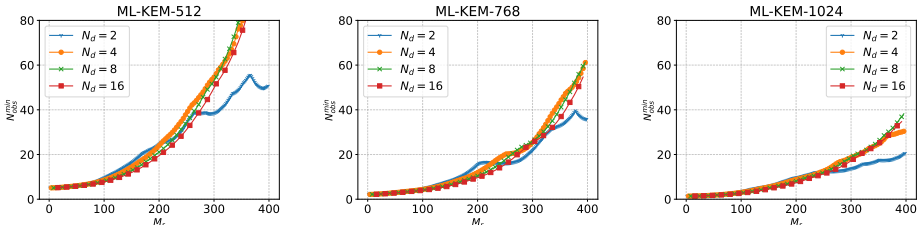


Figure 8: Evaluation results of N_{obs}^{min} (the number of observations required to recover a secret key coefficient) with the PKM technique.

5 Experimental Validation

To validate the theoretical estimation of the effect of RIC presented in Section 4, we conducted DFO-based and PCO-based attacks on a RIC implementation and empirically determined the number of observations required for successful key recovery. Additionally, we implemented the RIC countermeasure in C and evaluated its performance overhead relative to existing implementations.

5.1 DFO-based attack experiment

We executed DFO-based attacks against ML-KEM-512 with the RIC countermeasure and empirically validated whether the number of required inequalities matches the estimate given in Section 4.1. Although in practice an attacker must obtain inequalities in $\mathbf{Ax}[j]$ from side-channel observations, we omitted side-channel measurements in our experiments and instead derived the inequalities directly from the computed values of w'. This setup corresponds to an idealized attacker with zero observation errors. To recover the secret key from the obtained inequalities, we used the error-tolerant inequality solver proposed by Hermelink et al. [HMS+23], which combines belief propagation (BP) with lattice reduction. A key feature of this solver is the use of *check nodes* in the factor graph used in the BP algorithm, enabling key recovery while accounting for the ambiguity inherent in the inequalities. In our experiments, no errors occurred in the inequalities arising from side-channel observations; however, errors introduced by RIC occurred with a probability given by $p_e = P(X = 0) \cdot P(Y = 1|X = 0) + P(X = 1) \cdot P(Y = 0|X = 1)$. Prior research [RPJ+24] indicated that the greedy algorithm can outperform BP in certain scenarios; however, a more recent study [HSMP24] showed that BP is generally optimal for noisy inequalities, despite its higher computational cost. Accordingly, we employed the solver proposed by Hermelink et al. [HMS+23], which first runs the BP algorithm, then transforms the unrecorded key coefficients into a low-dimensional closest vector problem (CVP), and achieves full-key recovery by solving this problem using a lattice-reduction method such as the blockwise Korkine–Zolotarev (BKZ) algorithm. In this study, following Hermelink et al.'s evaluation, we assessed the attack efficiency using the block size β of the lattice reduction problem solved by BKZ as a metric. In the original study [HMS+23], an attack is considered successful if $\beta < 70$.

Figure 9 presents the results of key recovery attacks conducted using the solver proposed by Hermelink et al. [HMS+23] against protected implementations with $M_r = 25$, 50, and

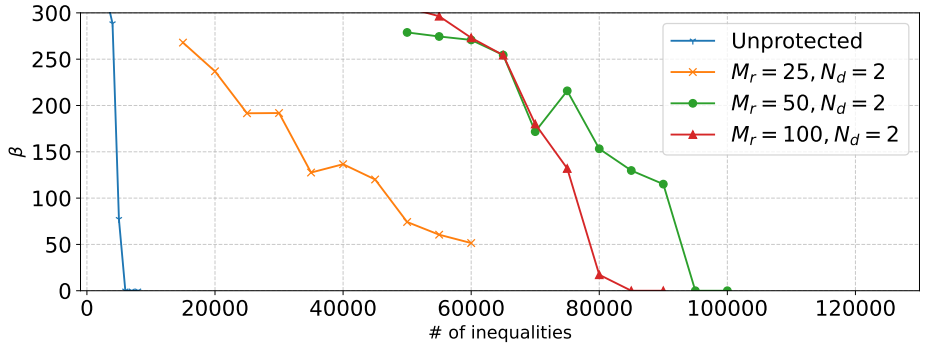


Figure 9: The average remaining security β of an ML-KEM-512 after DF-oracle-based attacks using the BP method proposed by Hermelink et al. [HMS+23]

Table 3: Comparison of the expected and empirical numbers of inequalities with correct probability p for successful key recovery

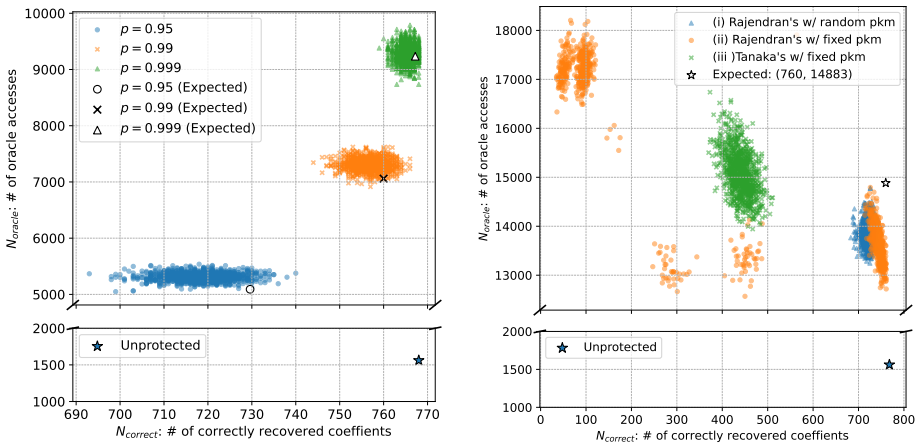
	$p = 1.0$	$p = 0.9$	$p = 0.87$	$p = 0.8$	$p = 0.77$	$p = 0.68$
Delvaux [Del22]	9000	34000	N/A	Unsolved	N/A	N/A
Hermelink et al. [HMS+23]	5500	12000	N/A	24000	N/A	N/A
This study	5500	N/A	70000	N/A	100000	80000
Expected $5500/(1 - H_2(p))$	5500	10355	12427	19777	23705	55385

100 ($N_d = 2$). For each number of inequalities, five key-recovery attempts were executed on randomly chosen secret keys, and the figure plots the average BKZ block size β. We used ciphertext queries with (approximately) $\omega = 832$ to maximize the key-recovery efficiency[4]. In this setting, the countermeasure gain $G = 1/C$ is 2.19, 4.31, and 10.07 for $M_r = 25$, 50, and 100, respectively. As shown in the figure, the number of inequalities required for key recovery increases with larger M_r, indicating that the proposed method provides a strong countermeasure effect. However, considering that the key-recovery attack against the unprotected implementation succeeds with 5500 inequalities, the experimental results deviate significantly from the number of inequalities predicted by $5500 \cdot G$: 12045, 23705, and 55385 for $M_r = 25$, 50, and 100, respectively. Hermelink et al.[HMS+23] also reported similar results. This is because the estimate based on the channel capacity provides only an approximate lower bound on the number of inequalities required for an attack. One possible explanation is the lack of independence among the inequalities. Because the amount of information derived from the channel capacity does not account for the overlap of information among inequalities, the actual information obtained becomes smaller when the inequalities are not independent.

Table 3 summarizes the expected and empirical numbers of inequalities with the correct probability p for a successful key recovery. Our experimental settings of $M_r = 25$, 50, and 100 ($N_d = 2$) correspond to $p = 0.87$, 0.77, and 0.68, respectively. Under the symmetric channel condition with $\omega = 832$, the expected number is the same as that in Hermelink et al. [HMS+23], $5500 \cdot G = \frac{5500}{1-H_2(p)}$. Nevertheless, our experimental results indicate that, for comparable values of p, a significantly larger number of inequalities is required than those reported by Hermelink et al. [HMS+23]. Hermelink et al. achieved successful key recovery using 24000 inequalities with a correct probability $p = 0.8$, whereas our experiments required approximately three times more inequalities, even at a higher $p = 0.87$.

This difference can be attributed to the fact that the types of errors introduced by our RIC countermeasures differ from those introduced in the experiments by Hermelink et

[4]We used $V_e = 832$ and $b \in [-5, 5]$ with the `-max-delta-v 5` option.



(a) W/o PKM (b) W/ PKM

Figure 10: Results of PCO-based attack experiments w/o and w/ PKM. For countermeasure parameters $M_r = 209$ and $N_d = 2$, the attacker uses the optimal attack parameters U_i and V_i. We performed 1,000 attack trials for each situation.

al. [HMS+23]. In the experiments by Hermelink et al. [HMS+23], errors were introduced uniformly at random among the inequalities satisfying $\mathbf{Ax}[j] \lessgtr b[j]$, whereas in our proposed method, errors were introduced only into the inequalities where $\mathbf{Ax}[j]$ is close to zero. This implies that our method selectively introduces uncertainty into inequalities carrying greater information, thereby forcing an attacker to obtain more inequalities for key recovery. As M_r increases, this effect gradually diminishes because $\mathbf{Ax}[j] \sim \mathcal{N}(0, 51^2)$, and thus the deviation from the theoretical value is expected to become smaller. In fact, the deviation from the theoretical estimate was roughly $\frac{70000}{12427} = 5.63$ times for $M_r = 25$, whereas it decreases to approximately $\frac{80000}{55385} = 1.44$ times for $M_r = 100$. These results indicate that our proposed countermeasure exhibits better-than-expected effectiveness even for small M_r, while also suggesting the need for a tighter security estimation.

5.2 PCO-based attacks

We implemented ML-KEM-768 with RIC in Python and performed PCO-based attack experiments to empirically evaluate key recovery efficiency. In each experiment, we conducted multiple attack trials and recorded (i) the number of correctly recovered secret keys $N_{correct}$ and (ii) the number of oracle accesses (ciphertext queries) used in those trials, N_{oracle}. We assume that the attacker has full knowledge of the countermeasure parameters M_r and N_d, and for each hypothesis H_i, used the optimal attack parameters U_i and V_i.

5.2.1 Experimental results without public-key mask

Figure 10(a) shows the experimental results without the public-key mask under three target-odds conditions: $O_{goal} = \log \frac{p}{1-p}$, with $p = 0.95$, 0.99, and 0.999. For each setting, 1000 attack trials were conducted using randomly chosen secret keys. The countermeasure parameters were set to $M_r = 209$ and $N_d = 2$. Under these settings, the predicted number of oracle accesses, $\hat{N}_{oracle} = 768 \cdot N_{obs}^{min}$ was 5091, 7059, and 9235 for the three target odds conditions, respectively. The attacker was assumed to use the optimal attack parameters $(U_0 = 211, V_0 = 2497), (U_1 = 208, V_1 = 416), (U_2 = 107, V_2 = 1497)$, and $(U_3 = 107, V_3 = 832)$.

The experimental results are generally consistent with the theoretical predictions, although both N_{oracle} and $N_{correct}$ show slight bias in a direction unfavorable to the attacker. As p decreases, the uncertainty of each hypothesis increases, resulting in a higher variance in $N_{correct}$. Conversely, for larger p, the variance of N_{oracle} increases. This is because when p is large, most secret key coefficients can be uniquely determined (as D_{KL} diverges), whereas a small number of coefficients for which D_{KL} does not diverge remain uncertain, requiring additional observations to eliminate this uncertainty. The figure shows that almost all secret-key coefficients can be recovered at the target odds $\log \frac{0.999}{1-0.999}$. Compared with the unprotected implementation, where all secret-key coefficients can be recovered with approximately $768 \times 2.03 = 1559$ observations, the proposed countermeasure with $M_r = 209$ and $N_d = 2$ achieves a $\frac{9235}{1559} = 5.9$-fold improvement in resistance against PCO-based attacks. We also conducted experiments with other countermeasure parameters M_r and N_d, which exhibited trends similar to those in Figure 10(a), with results closely matching theoretical predictions.

5.2.2 Experimental results with public-key mask

Next, we experimentally evaluated the effectiveness of RIC against PCO-based attacks when the attacker used ciphertext queries with a public-key mask (PKM). We assumed that the attacker used the simplest form of PKM with scaling factor $\alpha = 1$ and rotation factor $\beta = 0$, such that $\mathbf{M}_{\mathrm{pk}} = 1 \cdot X^0 \cdot (\mathbf{A}[\gamma], \mathbf{t}[\gamma])$. When other forms of PKM are employed, particularly those with a larger α, the distribution in Equation 25 becomes broader, reducing the attack's effectiveness.

While we estimated the number of observations required for key recovery in a PKM PCO-based attack in Figure 8, we assumed that a fresh PKM was generated uniformly at random for each query. However, an attacker may instead reuse a single PKM across all queries, and the attack performance may differ depending on the ciphertext query strategy. Specifically, the Rajandran-style ciphertext in [RRD+23] may exhibit different behavior from Tanaka-style ciphertext in [UXT+22].

In the Rajandran-style attack, the attacker focuses on a specific message-polynomial coefficient $w'[0]$ across all queries. When a single PKM $\mathbf{M}_{\mathrm{pk}}$ is fixed, the noise component originating from it remains constant for every query. Consequently, if the chosen $\mathbf{M}_{\mathrm{pk}}$ happens to induce a small noise value by chance, the attacker may succeed with fewer observations than estimated. In contrast, the Tanaka-style attack focuses on different message-polynomial coefficients $w'[j]$ across queries. Therefore, even if the same PKM $\mathbf{M}_{\mathrm{pk}}$ is reused, the PKM-induced noise component applied to observations varies across queries. As a result, the attack performance using Tanaka-style ciphertext remains stable across different fixed $\mathbf{M}_{\mathrm{pk}}$ values and is expected to be closer to our estimation.

Figure 10(b) shows the PCO-based attack results for three settings: (i) Rajendran-style attack with a randomly generated PKM: $\mathbf{M}_{pk} = 1 \cdot X^{\beta} \cdot (\mathbf{A}[\gamma], \mathbf{t}[\gamma])$, $(\beta \in [0, 511], \gamma \in [0, 2])$ for each query. (ii) Rajendran-style attack with a fixed PKM for each trial. (iii) Tanaka-style attack with a fixed PKM for each trial. For each setting, we performed key-recovery experiments on 1000 different secret keys under $M_r = 209$, $N_d = 2$, and $p = 0.99$. The attacker was assumed to have full knowledge of the countermeasure parameters M_r and N_d, and experiments were conducted using the optimal attack parameters (U_i, V_i) for the ith hypothesis H_i: $(U_0 = 202, V_0 = 624), (U_1 = 218, V_1 = 416), (U_2 = 189, V_2 = 832)$, and $(U_3 = 247, V_3 = 624)$.

As expected, the experimental results for setting (i) were generally consistent with theoretical predictions $(= 768 \cdot 16.38 = 14883)$, achieving 99% key recovery with approximately ten times more observations than the unprotected implementation. The deviation from the predicted values was attributed to several approximations used in calculating the distribution $\mathcal{D}_{pkm}$. In contrast, results for setting (ii) showed large variations forming some distinct clusters, presumed to correspond to different noise ranges originating from

a fixed $\mathbf{M}_{\mathrm{pk}}$. Although some cases within setting (ii) achieved more than 760 recovered coefficients with fewer observations than the expected value of 14883, their impact on overall key recovery performance was limited to approximately 10–20%. Results for setting (iii) showed smaller variance than those of setting (ii), as expected, but only approximately 430 coefficients were successfully recovered, indicating a significantly lower efficiency than in setting (i). This can be attributed to how PKM is applied: in setting (i), a new PKM is randomly generated for each query, whereas in setting (iii), the same PKM-induced noise is reused across multiple queries required to recover a particular secret-key coefficient leading to correlated noise among queries, and reduced attack efficiency.

In summary, setting (i), which follows the theoretical model, demonstrates good key-recovery efficiency, whereas settings (ii) and (iii) tend to exhibit lower efficiencies because of deviations from the theoretical model. However, these results are based on the assumption that the theoretical model corresponding to (i) is valid. If theoretical models for (ii) and (iii) can be established, their efficiency could potentially improve. Moreover, the attack parameters (U_i, V_i) used in this experiment were optimized for setting (i); however, more robust choices of (U_i, V_i) might exist when considering the characteristics of settings (ii) and (iii). For example, some noise-robust PCO attacks has recently been proposed in [HMT25, PRI25, LCS+24]. A detailed analysis of these aspects is left for future work.

6 Discussion

6.1 Concern on the DFR degradation

While conventional SCA countermeasures improve SCA resistance at the cost of increased computational overhead, RIC achieves improved SCA resistance without increasing computation by sacrificing the decryption failure probability p_f. We assume that $p_f \leq 2^{-80}$ is acceptable, and evaluate attacks under parameters satisfying this condition (for ML-KEM-768, $M_r = 210, N_d = 2$). In this section, we discuss the impact of $p_f \approx 2^{-80}$.

6.1.1 Impact on device operation

Since RIC is intended to be implemented on low-cost devices such as smart cards, we assume that the maximum number of queries executed by a single device is $T_{query} = 2^{40}$. This corresponds to continuously executing ML-KEM once per millisecond (i.e., at 1 kHz) for 34 years, and is therefore a conservative assumption for most devices that use KEM only intermittently[5]. In this case, the probability that a decryption failure occurs on a single device is $1 - (1 - 2^{-80})^{2^{40}} \approx 2^{-40}$, which is almost negligible.

However, in mass-produced devices where even a single decryption failure across all devices is unacceptable, $p_f \approx 2^{-80}$ may not be acceptable. For example, if 2^{40} devices use RIC, the total number of queries becomes 2^{80}, resulting in a probability of about 63% that at least one decryption failure occurs. In such mass-production scenarios, it is necessary to implement handling mechanisms such as detecting decapsulation failures and retrying key exchange. On the other hand, devices targeted by SCA typically operate with a much smaller T_{query}, so these estimates may be overly conservative. For example, EMV cards limit the number of queries to 2^{16} using a 16-bit monotonic counter called the Application Transaction Counter (ATC), and thus $p_f \approx 2^{-80}$ could be sufficiently practical depending on the application.

[5]According to the pqm4 [mup] benchmark, ML-KEM-768 decapsulation takes about 700k cycles on Cortex M4, which corresponds to 3.5ms at 200MHz.

6.1.2 Security against black-box attacks

While RIC improves the resistance against SCA-CCA (that is gray-box attacks), might degrade the resistance against black-box attacks. If an unmodified ciphertext results in a decryption failure, a decapsulation failure, which is observable even in black-box setting, leads to leakage of key information. Under an optimistic single-target evaluation assuming $T_{query} = 2^{40}$, the probability of observing a decryption failure remains negligible even with $p_f \approx 2^{-80}$, and thus the scheme can be considered secure in black-box model. However, recent series of works by D'Anvers et.al, [DVV18, DRV20, DB22] have investigated more practical black-box DF attacks. We discuss their impact on $p_f \approx 2^{-80}$ below.

Decryption Failure boosting In [DVV18], D'Anvers et al. proposed a method to boost the DFR by selecting and querying only ciphertexts that are likely to cause decryption failures (weak ciphertexts) from randomly generated ciphertexts. Recall that the message polynomial w is given by

$$\begin{aligned} w &= e_2 + \Delta v + \mathbf{e}^\top \cdot \mathbf{y} - \mathbf{s}^\top \cdot \mathbf{e}_1 - \mathbf{s}^\top \cdot \Delta\mathbf{u} \\ &= \mathbf{A}\mathbf{x} + b \end{aligned}$$

Here, $\mathbf{x} = (\mathbf{s}, \mathbf{e})^\top$ represents the secret information, and $\mathbf{A} = (\mathbf{y}, -(\mathbf{e}_1 + \Delta\mathbf{u}))$ and $b = -e_2 - \Delta v$ are ciphertext components that can be chosen by the attacker. The attacker first generates ciphertexts $ct = (\mathbf{A}, b)$ randomly, and classifies a ciphertext as weak if the probability of causing a failure $P[F|ct]$ exceeds a threshold f_t, and queries the oracle only with such weak ciphertexts. $P[F|ct]$ is defined as the expected decryption failure probability over all secret keys for a given ciphertext:

$$P[F|ct] := \sum_{\mathbf{x}} P[\|\mathbf{A}\mathbf{x} + b\|_\infty > q_t \mid \mathbf{x}] \cdot P[\mathbf{x}].$$

Intuitively, larger $\mathbf{A}\mathbf{x}$ tends to increase the probability of failure; see [DVV18] for details on how to concretely compute it. When $\mathbf{A}\mathbf{x}$ causing a failure is obtained, it is known that these vectors are aligned (or anti-aligned) with $\mathbf{x}$, which allows narrowing down the candidate space of $\mathbf{x}$. The amount of information about $\mathbf{x}$ obtained from this process is also analyzed in [DVV18].

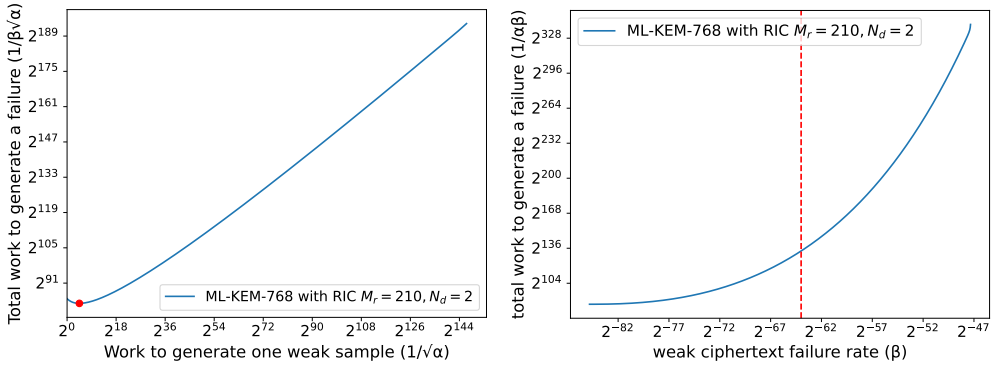
It is shown in [DVV18] that a single decryption failure provides very limited information about the secret key, and that multiple (tens of) failures are required to significantly reduce the entropy of the secret key. However, a follow-up work by D'Anvers et al. [DRV20] shows that once a weak ciphertext that causes a failure is found, the approximate direction of the secret key can be inferred, also allowing faster failure boosting. Therefore, the cost of observing the first decryption failure dominates the attack, and for simplicity, we evaluate only the cost of finding the first failure. In practice, especially in multi-target settings [DB22], the cost of finding subsequent failures is not negligible, so this constitutes a worst-case security evaluation.

The cost of attacks using failure boosting is evaluated using the following two metrics:

$$\alpha_{f_t} = \sum_{\forall ct : P[F|ct] > f_t} P[ct],$$

$$\beta_{f_t} = \frac{\sum_{\forall ct : P[F|ct] > f_t} P[ct] \cdot P[F|ct]}{\alpha_{f_t}}$$

Here, α_{f_t} is the probability of weak ciphertexts, meaning that the attacker needs on average $\alpha_{f_t}^{-1}$ work to find one weak ciphertext. An attacker with access to a quantum computer can reduce this cost to $\sqrt{\alpha_{f_t}}^{-1}$ using Grover's algorithm. β_{f_t} is the probability of observing a



(a) As a function of α. (b) As a function of β.

Figure 11: Total work to generate a failure $\sqrt{\alpha_{f_t}}^{-1}\beta_{f_t}^{-1}$

decryption failure when querying only weak ciphertexts. Using these, the work required for a (quantum) attacker to observe one decryption failure with probability $1 - e^{-1} \approx 0.63$ can be expressed as $\sqrt{\alpha_{f_t}}^{-1}\beta_{f_t}^{-1}$.

Single-target attack on RIC using failure boosting When using RIC with $M_r = 210$, the effect of failure boosting can be estimated by computing $P[F|ct]$ with $q_t = 832 - 210 = 622$. Strictly speaking, due to the effect of N_d, coefficients exceeding 622 do not cause failures with probability 1, but this provides a good worst-case estimate independent of N_d. We evaluate the failure boosting effect $\sqrt{\alpha_{f_t}}^{-1}\beta_{f_t}^{-1}$ for RIC using the tool [dan], and show the results in Figure 11a. Since α_{f_t} and β_{f_t} are both functions of f_t and have a trade-off relationship, it is rational for the attacker to choose parameters minimizing $\sqrt{\alpha_{f_t}}^{-1}\beta_{f_t}^{-1}$.

As shown at the red point in the figure, the minimum is achieved at very small $\sqrt{\alpha}_{f_t}^{-1} = 2^{4.5}$, where $\beta_{f_t}^{-1} = 2^{78.44}$ oracle accesses are required. Since the NIST call for proposals limits the number of oracle queries under chosen-ciphertext attacks to $T_{query} = 2^{64}$, DF-based attacks under this condition are not effective.

Next, Figure 11b shows $\sqrt{\alpha}_{f_t}^{-1}\beta_{f_t}^{-1}$ as a function of β. From this figure, it can be seen that approximately 2^{128} quantum work is required to succeed under $T_{query} = 2^{64}$. Although this does not satisfy NIST quantum security level 3, it can still be considered secure in many practical scenarios, at least for the foreseeable future. Furthermore, as discussed in Section 6.1.1, the T_{query} of devices implementing RIC is expected to be smaller, in which case more than 130-bit quantum security can be achieved.

Multi-target attack on RIC using failure boosting Finally, we consider the impact of extending to the multi-target setting. In [DB22], the multi-target effect is modeled simply as an increase in the number of queries. Therefore, for T_{dev} devices, security is maintained when $\beta^{-1} > T_{dev} \cdot T_{query}$. From the results in Figure 11b, at least β^{-64} is required to achieve NIST security level 1; thus, if $T_{query} = 2^{40}$, the system is secure up to 2^{24} devices. Since T_{query} and T_{dev} are in a trade-off relationship, applications with smaller T_{query} can claim security for larger T_{dev}. As RIC is intended for low-cost devices where T_{query} can be limited, it is considered sufficiently practical under such conditions.

7 Conclusion

We proposed RIC, a lightweight and masking-friendly countermeasure against SCA-CCAs on ML-KEM. The proposed method randomizes invalid coefficients during decryption,

increasing the difficulty of key recovery for an adversary by introducing probabilistic errors.

Through theoretical analysis, we modeled the proposed scheme as a binary asymmetric channel and demonstrated that the mutual information between the secret key and side-channel leakage is significantly reduced. The results indicate that an attacker must perform more than an order of magnitude more oracle accesses to recover the key when realistic parameters are used, while maintaining an acceptable DFR of approximately 2^{-80}. Our experimental validation confirmed that RIC achieves strong resistance to both DFO- and PCO-based attacks, with only approximately 1% performance overhead compared to an unprotected implementation.

These results demonstrate that introducing randomness into the decryption function can effectively balance the performance and security of post-quantum cryptographic implementations. Future work will include integrating the proposed technique with other lightweight countermeasures, such as shuffling and dummy operations, and evaluating its effectiveness against higher-order and fault attacks.

Acknowledgment

This paper is based on results obtained from a project, JPNP24003, commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

References

[ABB+24] Estuardo Alpirez Bock, Gustavo Banegas, Chris Brzuska, Lukasz Chmielewski, Kirthivaasan Puniamurthy, and Milan Sorf. Breaking DPA-protected Kyber via the pair-pointwise multiplication. In Christina Pöpper and Lejla Batina, editors, *ACNS 24International Conference on Applied Cryptography and Network Security, Part II*, volume 14584 of *LNCS*, pages 101–130. Springer, Cham, March 2024.

[BGR+21] Joppe W. Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine van Vredendaal. Masking kyber: First- and higher-order implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):173–214, Aug. 2021.

[CGMZ23] Jean-Sébastien Coron, François Gérard, Simon Montoya, and Rina Zeitoun. High-order polynomial comparison and masking lattice-based encryption. *IACR TCHES*, 2023(1):153–192, 2023.

[dan] danversjp. Pqcrypto-decryption-failures. https://github.com/KULeuven-COSIC/PQCRYPTO-decryption-failures.

[DB22] Jan-Pieter D'Anvers and Senne Batsleer. Multitarget decryption failure attacks and their application to Saber and Kyber. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part I*, volume 13177 of *LNCS*, pages 3–33. Springer, Cham, March 2022.

[Del22] Jeroen Delvaux. Roulette: A diverse family of feasible fault attacks on masked Kyber. *IACR TCHES*, 2022(4):637–660, 2022.

[DNGW23] Elena Dubrova, Kalle Ngo, Joel Gärtner, and Ruize Wang. Breaking a fifth-order masked implementation of crystals-kyber by copy-paste. In *Proceedings of the 10th ACM Asia Public-Key Cryptography Workshop*, APKC '23, page 10–20, New York, NY, USA, 2023. Association for Computing Machinery.

[DRV20] Jan-Pieter D'Anvers, Mélissa Rossi, and Fernando Virdia. (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 3–33. Springer, Cham, May 2020.

[DVV18] Jan-Pieter D'Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. On the impact of decryption failures on the security of LWE/LWR based schemes. Cryptology ePrint Archive, Report 2018/1089, 2018.

[HDH+25] Fan Huang, Xiaolin Duan, Chengcong Hu, Mengce Zheng, and Honggang Hu. Mulleak: Exploiting multiply instruction leakage to attack the stack-optimized kyber implementation on cortex-m4. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):23–68, Mar. 2025.

[HHP+21] Mike Hamburg, Julius Hermelink, Robert Primas, Simona Samardjiska, Thomas Schamberger, Silvan Streit, Emanuele Strieder, and Christine van Vredendaal. Chosen ciphertext k-trace attacks on masked CCA2 secure Kyber. *IACR TCHES*, 2021(4):88–113, 2021.

[HME24] Xunyue Hu, Quentin L. Meunier, and Emmanuelle Encrenaz. Blind-folded: Simple power analysis attacks using data with a single trace and no training. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):475–496, Dec. 2024.

[HMS+23] Julius Hermelink, Erik Mårtensson, Simona Samardjiska, Peter Pessl, and Gabi Dreo Rodosek. Belief propagation meets lattice reduction: Security estimates for error-tolerant key recovery from decryption errors. *IACR TCHES*, 2023(4):287–317, 2023.

[HMT25] Julius Hermelink, Erik Mårtensson, and Maggie Tran. Noise-tolerant plaintext-checking oracle attacks – a soft-analytic approach applied to ML-KEM. Cryptology ePrint Archive, Paper 2025/1496, 2025.

[HNPS24] Julius Hermelink, Kai-Chun Ning, Richard Petri, and Emanuele Strieder. The insecurity of masked comparisons: SCAs on ML-KEM's FO-transform. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2430–2444. ACM Press, October 2024.

[HSMP24] Julius Hermelink, Silvan Streit, Erik Mårtensson, and Richard Petri. A generic framework for side-channel attacks against LWE-based cryptosystems. Cryptology ePrint Archive, Paper 2024/1211, 2024.

[KdG21] Alexandre Karlov and Natacha Linard de Guertechin. Power analysis attack on Kyber. Cryptology ePrint Archive, Report 2021/1311, 2021.

[LCS+24] Jinnuo Li, Chi Cheng, Muyan Shen, Peng Chen, Qian Guo, Dongsheng Liu, Liji Wu, and Jian Weng. Grafted trees bear better fruit: An improved multiple-valued plaintext-checking side-channel attack against kyber. Cryptology ePrint Archive, Paper 2024/1174, 2024.

[MBB+22] Catinca Mujdei, Arthur Beckers, Jose Bermundo, Angshuman Karmakar, Lennert Wouters, and Ingrid Verbauwhede. Side-channel analysis of lattice-based post-quantum cryptography: Exploiting polynomial multiplication. Cryptology ePrint Archive, Report 2022/474, 2022.

[mup] mupq. pqm4. https://github.com/mupq/pqm4.

[(NS] National Security Agency (NSA). Announcing the commercial national security algorithm suite 2.0. https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF.

[oSN24a] National Institute of Standards and Technology (NIST). Module-lattice-based digital signature standard. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf, 2024.

[oSN24b] National Institute of Standards and Technology (NIST). Module-lattice-based key-encapsulation mechanism standard. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf, 2024.

[pc] pq crystals. security-estimates. https://github.com/pq-crystals/security-estimates.

[PQC] PQClean. Pqclean. https://github.com/PQClean/PQClean.

[PRI25] Paco Poilbout, Thomas Roche, and Laurent Imbert. Handling noisy plaintext checking oracles with SPiRiT. Cryptology ePrint Archive, Paper 2025/2045, 2025.

[RCDB24] Prasanna Ravi, Anupam Chattopadhyay, Jan Pieter D'Anvers, and Anubhab Baksi. Side-channel and fault-injection attacks over lattice-based post-quantum schemes (kyber, dilithium): Survey and new results. *ACM Trans. Embed. Comput. Syst.*, 23(2), Mar 2024.

[RPJ+24] Prasanna Ravi, Thales Paiva, Dirmanto Jap, Jan-Pieter D'Anvers, and Shivam Bhasin. Defeating low-cost countermeasures against side-channel attacks in lattice-based encryption A case study on Crystals-Kyber. *IACR TCHES*, 2024(2):795–818, 2024.

[RRD+23] Gokulnath Rajendran, Prasanna Ravi, Jan-Pieter D'Anvers, Shivam Bhasin, and Anupam Chattopadhyay. Pushing the limits of generic side-channel attacks on LWE-based KEMs - parallel PC oracle attacks on Kyber KEM and beyond. *IACR TCHES*, 2023(2):418–446, 2023.

[SKL+20] Bo-Yeon Sim, Jihoon Kwon, Joohee Lee, Il-Ju Kim, Taeho Lee, Jaeseung Han, Hyojin Yoon, Jihoon Cho, and Dong-Guk Han. Single-trace attacks on the message encoding of lattice-based KEMs. Cryptology ePrint Archive, Report 2020/992, 2020.

[TUX+23] Yutaro Tanaka, Rei Ueno, Keita Xagawa, Akira Ito, Junko Takahashi, and Naofumi Homma. Multiple-valued plaintext-checking side-channel attacks on post-quantum KEMs. *IACR TCHES*, 2023(3):473–503, 2023.

[UXT+22] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. Curse of re-encryption: A generic power/EM analysis on post-quantum KEMs. *IACR TCHES*, 2022(1):296–322, 2022.

[WBD24] Ruize Wang, Martin Brisfors, and Elena Dubrova. A side-channel attack on a higher-order masked CRYSTALS-Kyber implementation. In Christina Pöpper and Lejla Batina, editors, *ACNS 24International Conference on Applied Cryptography and Network Security, Part III*, volume 14585 of *LNCS*, pages 301–324. Springer, Cham, March 2024.

[XPR+22] Zhuang Xu, Owen Pemberton, Sujoy Sinha Roy, David Oswald, Wang Yao, and Zhiming Zheng. Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of kyber. *IEEE Transactions on Computers*, 71(9):2163–2176, 2022.

Table 4: Distribution of a coefficient of $\Delta\mathbf{u}$ and $\mathbf{s}[\cdot,\cdot] \times \Delta\mathbf{u}[\cdot,\cdot]$ (ML-KEM-768).

$\Delta\mathbf{u}[\cdot,\cdot]$	-2	-1	0	1	2
$P(\Delta\mathbf{u}[\cdot,\cdot])$	$\frac{129}{3329}$	$\frac{1024}{3329}$	$\frac{1024}{3329}$	$\frac{1024}{3329}$	$\frac{128}{3329}$

$\mathbf{s}[\cdot,\cdot] \times \Delta\mathbf{u}[\cdot,\cdot]$	-4	-2	-1	0	1	2	4
$P(\mathbf{s}[\cdot,\cdot] \times \Delta\mathbf{u}[\cdot,\cdot])$	$\frac{257}{53264}$	$\frac{3076}{53264}$	$\frac{8192}{53264}$	$\frac{30214}{53264}$	$\frac{8192}{53264}$	$\frac{3076}{53264}$	$\frac{257}{53264}$

Appendix

A Derivation of the distribution of r_{pkm}

Assuming that public keys are uniformly distributed, a coefficient of $\Delta\mathbf{u}_M$ follows the distribution given in Table 4. This table also presents the distribution of $\mathbf{s}[\cdot,\cdot] \times \Delta\mathbf{u}[\cdot,\cdot]$, which is the product of the coefficient of $\Delta\mathbf{u}_M$ and the coefficient of the secret key $\mathbf{s}$. This product has a mean of zero and a variance of 0.924. As a coefficient of the result of the polynomial multiplication $\mathbf{s}[\cdot] \times \Delta\mathbf{u}_M[\cdot]$ can be regarded as the sum of 256 samples from $P(\mathbf{s}[\cdot,\cdot] \times \Delta\mathbf{u}[\cdot,\cdot])$, it can be approximated by a normal distribution $\mathcal{N}(0, 236)$ using the central limit theorem. Similarly, the coefficient of the inner product $\langle \mathbf{s}, \Delta\mathbf{u}_M \rangle$ can be regarded as the sum of k samples from $\mathcal{N}(0, 236)$ and therefore follows the distribution $\mathcal{N}(0, 236k)$. The coefficients of Δv_M are uniformly distributed over the range $[-3328/2^{d_v+1}, 3328/2^{d_v+1}]$. Consequently, the distribution of $r_{pkm} = \Delta v_M - \langle \Delta\mathbf{u}_M, \mathbf{s} \rangle$ is given by the sum of the uniformly weighted normal distributions:

$$\mathcal{D}_{\mathrm{pkm}} = \frac{2^{d_v}}{3328 + 2^{d_v}} \cdot \sum_{i=-3328/2^{d_v+1}}^{3328/2^{d_v+1}} \mathcal{N}(i, 236k). \tag{29}$$